

HANDS ON TIPS FOR CRACKING THE INTERVIEW

C++

Interview Questions



C++

Interview Questions

Authored by:

Kogent Learning Solutions Inc.

Published by:



©Copyright by Dreamtech Press, 19-A, Ansari Road, Daryaganj, New Delhi-110002

This book may not be duplicated in any way without the express written consent of the publisher, except in the form of brief excerpts or quotations for the purposes of review. The information contained herein is for the personal use of the reader and may not be incorporated in any commercial programs, other books, databases, or any kind of software without written consent of the publisher. Making copies of this book or any portion for any purpose other than your own is a violation of copyright laws.

Limits of Liability/disclaimer of Warranty: The author and publisher have used their best efforts in preparing this book. The author make no representation or warranties with respect to the accuracy or completeness of the contents of this book, and specifically disclaim any implied warranties of merchantability or fitness of any particular purpose. There are no warranties which extend beyond the descriptions contained in this paragraph. No warranty may be created or extended by sales representatives or written sales materials. The accuracy and completeness of the information provided herein and the opinions stated herein are not guaranteed or warranted to produce any particular results, and the advice and strategies contained herein may not be suitable for every individual. Neither Dreamtech Press nor author shall be liable for any loss of profit or any other commercial damages, including but not limited to special, incidental, consequential, or other damages.

Trademarks: All brand names and product names used in this book are trademarks, registered trademarks, or trade names of their respective holders. Dreamtech Press is not associated with any product or vendor mentioned in this book.

ISBN: 978-81-7722-804-5

ISBN: 978-93-5004-918-1 (ebk)

Edition: 2011

Printed at: Himal Impressions.

Table of Contents

Introduction	v
Chapter 1: Constructors / Destructors	1
Chapter 2: Virtual Functions	25
Chapter 3: Inheritance	29
Chapter 4: Polymorphism	55
Chapter 5: Classes and Objects.....	57
Chapter 6: Memory Allocation / Deallocation	73
Chapter 7: Pointers.....	87
Chapter 8: Strings	91
Chapter 9: Overloading Operators.....	93
Chapter 10: Algorithms	99
Chapter 11: Friends	115
Chapter 12: Miscellaneous.....	121
Chapter 13: C and C++	137
Question Index.....	141
Glossary.....	153

Introduction

Congratulations on buying the C++ Interview Questions book! This book is designed especially for students and professionals who intend to prepare for interviews on C++. It is often difficult to search and obtain a book that gives an overview of important topics and some valuable concepts that you feel might be asked in an interview. Such a frenetic search normally consumes a lot of time and energy, as you need to explore huge volumes of books on C++ programming. We present this book, C++ Interview Questions, as a one-stop solution to your search for a quality book focusing on the probable interview questions on C++.

This book covers all the useful content necessary to clear an interview. It is compiled in an easy-to-understand and self-explanatory manner. This book gives a quick understanding of the C++ programming language and its in-depth features. In this book, code snippets and programs have been tested on a variety of C++ compilers, such as Turbo C++ and Borland. However, you need to add some additional lines of code to compile and execute the C++ programs, depending upon the compiler you use. The book is divided into following 13 chapters:

- *Chapter 1* —Constructors / Destructors, discusses about copy constructor, overload assignment operator, default constructor, conversion constructor, conversion operator, shallow copy, deep copy, zombie objects, delete operator, new operator, assignment operator, and named constructor.
- *Chapter 2* —Virtual Functions, covers concepts, such as definitions of virtual and pure virtual functions, pure virtual member function and virtual destructors.
- *Chapter 3* —Inheritance, introduces the concept of multiple inheritance, template, class template, function template, non-inheritance, static typing, dynamic binding, non-virtual member function, performance cost, overriding, upcasting, template container class, implementation, interface, abstract base classes, private inheritance, composition, void pointers,

protected inheritance, public virtual functions, protected virtual functions, leaf method and dreaded diamond.

- *Chapter 4* —Polymorphism, covers definition of polymorphism and Runtime Type Identification (RTTI).
- *Chapter 5* —Classes and Objects, introduces the definition of class and structures, encapsulation, implicit member functions of a class, container class, nested class, local class, derived class, static data members, and objects
- *Chapter 6* —Memory Allocation/ Deallocation, covers the Malloc() operator, new operator, memory alignment, dynamic allocation, two-dimensional array, throw statement, Microsoft Foundation Class (MFC).
- *Chapter 7* —Pointers, explores the concepts of dangling, smart and auto pointers, memory leak, pointers and reference.
- *Chapter 8* —Strings, describes the concept of type conversion.
- *Chapter 9* — Overloading Operators, covers topics, such as function overloading, operator overloading, the postfix operator ++(), pass by reference and pass by value.
- *Chapter 10* —Algorithms, includes description of a linked list, binary heap, traversal, insertion, and deletion in a binary search tree.
- *Chapter 11* —Friends, covers the Friend function, Print On() method and operator <<.
- *Chapter 12* —Miscellaneous, introduces the inline functions, scope resolution operator, debugging methods, C++ storage classes, storage qualifiers, const reference arguments, temporary variables, namespace, Iterator class, stack unwinding, pointer hopping and echo feature.
- *Chapter 13* —C and C++, contains information about name mangling, type-safe linkages and extern c, public members and private members.

Your feedback and suggestions are always welcome. Happy Job Hunting!!

1

Constructors / Destructors

1. Define what is constructor or ctor.

- A constructor is a method that is called when a new instance of an object of a class is created. It initializes all class members whenever you create an object of the class. It is a member function with the same name as its class. It may or may not take any parameters. It does not have any return type. For example, the compiler embeds a call to the constructor for each object when it is created. Suppose a class Z has been declared as follows:

```
class Z {  
    public:  
    Z(); // constructor for class Z  
};
```

We cannot declare a constructor as virtual or static, nor we can declare a constructor as const, volatile, or const volatile.

The short form of constructor is ctor.

2. Can we define a constructor as virtual in C++?

- No, we cannot define constructors as virtual because constructors have the same name as their classes and no two constructors of base-derived classes can have the same name. So, when we initialize an object of the base or derived class with the help of virtual constructors, the base constructor is invoked instead of the derived constructor. Therefore, it is not possible to define a constructor as virtual.

2 C++ INTERVIEW QUESTIONS

3. Can we declare a base-class destructor as virtual?

- Yes, we can declare a base-class destructor as virtual that makes all derived-class destructors as virtual even if they do not have the same name as base-destructor. The problem arises when the derived class's pointer refers to a base class. For this reason, the base class destructor should be declared as virtual so that the appropriate destructor is called on calling the delete method of the base class object.

4. How does a copy constructor differs from an overloaded assignment operator?

- A copy constructor uses the value of an argument to construct a new object. We can use an overload assignment operator to assign the value of an existing object of the same class to another existing object in that class.

5. Define two methods of constructor invocation.

- The following are the two ways to invoke a constructor:
 1. The compiler embeds a call to the constructor for each object that has got created. For example:

```
class A
{
    int x;
public: void setx(const int=0);
    int getx();
};
main(){
    A A1; // object declared-constructor called.
    A1.A(); // constructor called implicitly.
}
```

2. The constructor is called for each object that is created dynamically in the heap by the new operator. For example:

```
A *Aptr;
Aptr = new A;    // constructor called implicitly
                by compiler.
```

6. Define the process of error-handling in case of constructor failure.

- If the constructor does not have the return statement, then it indicates failure in handling the error by throwing an exception.

7. Explain the concept of copy constructor.

- Copy constructor is a special type of parameterized constructor. It copies one object to another. It is called when an object is created and equated to an existing object at the same time. An existing object is passed as a parameter to it.

For example:

```
A A1;          // default constructor called.  
A A2=A1;       //copy constructor called.  
Or  
A A2(A1);      //copy constructor called.  
Or  
A *Aptr = new A(A1);    //copy constructor  
called.
```

8. What is a default constructor?

- A zero-argument constructor or a constructor in which all the arguments have default values is called a default constructor.

For example:

```
A A1;          // default constructor called.
```

9. When are copy constructors called?

- A copy constructor is defined by the programmer or by compiler itself. A call to copy constructor is embedded under the following three conditions:

1. When an object is created and at the same time equated to some other existing object. For example:

```
A A1;          // default constructor called.  
A A2=A1;       //copy constructor called.  
Or  
A A2(A1);      //copy constructor called.  
Or
```

4 C++ INTERVIEW QUESTIONS

```
A *Aptr = new A(A1);    //copy constructor
                        called.
```

2. When an object is created without having the reference of the argument, then the copy constructor is called by default for the argument object.

For example:

```
A A1;
xyz(A1);

void xyz(A A2){
// definition of xyz()
}
```

3. When an object is created and at the same time equated to a call to a function that returns an object.

For example:

```
A xyz()
{
A A1;
//remaining definition of xyz();
return A1;
}
A A2=xyz();
```

10. Is it possible to pass an object of the same class in place of object reference to the copy constructor?

- Yes, when an object is created without the reference formal argument, then the copy constructor is called for the argument object. The object that is passed as a parameter to the function is then passed as a parameter to the copy constructor.

For example:

```
void xyz(A);
A A1;
xyz(A1);

void xyz(A A2){
// definition of xyz()
}
```

In this case, the copy constructor is called for A2 while A1 is passed as a parameter to the copy constructor.

11. Define a conversion constructor.

- A conversion constructor is a single argument constructor. It is used by the compiler to convert a type of object as an argument to a class type.

```
#include <iostream.h>
#include <conio.h>
class Y {
    int a, b;
    char* name;
public:
    Y(int i) { };
    Y(const char* n, int j = 0) { };
};

void add(Y) { };

int main() {

    // equivalent to
    // obj1 = Y(2)
    Y obj1 = 2;

    // equivalent to
    // obj2 = Y("somestring",0)
    Y obj2 = "somestring";

    // equivalent to
    // obj1 = Y(10)
    obj1 = 10;

    // equivalent to
    // add(Y(5))
    add(5);
    getch();
}
```

6 C++ INTERVIEW QUESTIONS

The above example has the following two conversion constructors:

- *Y(int i)*—Helps in the conversion of integers to objects of class Y
- *Y(const char* n, int j = 0)*—Helps in the conversion of pointers from strings to objects of the Y class

12. What is a conversion operator?

- A conversion operator converts an object of a class in which you define the function to an object of a different data type. A conversion operator uses the C++ operator keyword in its declaration. C++ uses public as an access specifier, which can be used in a class as a public method for specific data type conversions.

```
For example:
#include <iostream.h>
#include <conio.h>
struct segment_g
{
    int multiplication;
    int division;
public:
    segment_g( int m = 0, int d = 1 ):
multiplication( m ),
    division( d )
    { }
    operator double() const
    {
        return double( multiplication ) / double(
division );
    }
    friend ostream& operator << ( ostream& outs,
const segment_g& seg );
};
ostream& operator << ( ostream& outs, const
segment_g& seg )
{
    int m = seg.multiplication;
    int d = seg.division;
```

```
        if (d < 0)
        {
            m = -m;
            d = -d;
        }
        return outs << m << "/" << d;
    }
int main()
{
    segment_g t( 4, -7 );
    double b = t;
    cout << t << " = " << b << endl;
    getch();
    return 0;
}
```

In the preceding code snippet, operator double has no function assigned for the modification of the object, so it is better to make it constant.

13. Define the process of handling in case of destructor failure.

- In order to handle a failed destructor, you need to write a message to a log file; however, do not throw an exception. There is a rule in C++ that exception cannot be thrown from a destructor, which is called when the process of "stack unwinding" occurs in other exceptions. For example, if someone says throw waste files(), the stack frames between the throw waste files() and the catch (waste files) will get popped. This is known as stack unwinding. It is the process of destroying all the local objects related to those stack frames and calling destructors in case of throwing of an exception by one of those destructors. For example, if an object named Bar is thrown, then the C++ runtime system is in a neutral situation means either to avoid the Bar and end up in the catch (waste files) or ignore the function Foo and look for a catch (Bar) handler. It will call in the terminate () process to end the program.

14. What is a virtual destructor?

- Virtual destructors help in destroying objects without knowing their type. With the help of the virtual function mechanism, the appropriate destructor for the object is invoked. In the case of abstract classes, destructors can also be declared as pure virtual functions. For example, class A derives from class B. Then, on calling the derived class dynamically at the execution time, the destructor will first call the derived class that is class A, and then the base class that is class B.

It is important to note that the `virtual` keyword, when used with the destructor, ensures the calling of all the derived and base class destructors and therefore helps in the proper execution and closing of the program.

15. What is the sequence of destruction of local objects?

- The destructor is executed in the reverse order of a constructor. Firstly, the objects are constructed, methods are invoked on them, and then they are destructed. In the following example, a's destructor will be executed after b's destructor.

```
void abc()
{
    Cat a;
    Cat b;
}
```

16. Write about the sequence in which objects in an array are destructed.

- The destructor is executed in the reverse order of constructor. Firstly, the objects are constructed, methods are invoked on them, and then they are destructed.

In the following example, the order for destructors will be `b[19]`, `b[18]`, `b[17]`, `b[16]`, `b[15]`, `b[14]`, `b[13]`, `b[12]`, `b[11]`, `b[10]`:

```
void def()
{
    Bat b[20];
}
```

17. Is it possible to overload the destructor of a class?

- No, the destructor of a class cannot be overloaded. We can have only one destructor for a class, called `class name::~~destructor name()`. For example, for a class named `fruit`, the syntax is - `fruit::~~fruit()`. It never takes any parameters and it never returns anything.

We cannot pass parameters to the destructor anyway, since you never explicitly call a destructor.

18. Can you explicitly call a destructor on a local variable?

- No, the destructor is called when the block closes in which the local variable was created.

If a destructor is called explicitly twice on the same object, the result ends in an undefined behavior because the local variable gets destroyed when the scope ends.

19. What happens if a local variable is destroyed within its scope before a program is completely executed? Can a destructor be called if needed?

- No, the destructor cannot be called if needed. Let's understand the preceding statement with the help of an example. There is a local variable `n` of a class `Notebook`. Now, the destruction of the local variable `n` occurs before the execution of a program:

```
class Notebook
{
    int n;
    void bcd ()
    Notebook n;
    //...insert code that should execute when n is
    still in its scope...
    we want the side-effect of n's destructor here!
```

10 C++ INTERVIEW QUESTIONS

```
//...insert code that should execute after n is
destroyed...
}
```

On calling a destructor explicitly, an undefined behavior is occurred, because the destructor may invoke twice for the local variable. Firstly, on explicit calling and secondly automatic calling when the local variable destroys. Therefore, a destructor cannot be called explicitly if needed.

20. Explain how would you handle a situation where you cannot call the destructor of a local explicitly?

- To avoid the explicit calling of a destructor, the time extent of the local variable can be wrapped in an artificial block {...}:

```
void time()
{
{
Copy c;
...insert code that should execute when c is still
open...
} c's destructor will be called implicitly here!
...insert code here that should execute after c is
closed...}
```

21. What happens if the local variable cannot be wrapped in an artificial block?

- Most of the times, the time duration of wrapping local variables is limited in an artificial block ({...}). In other situations, a member function with similar effects can be added as a destructor instead of calling the destructor itself.

For example, in the case of class of File1, a close() method can be added. Usually, the close() method can be simply called by the destructor. Note that there is a need of marking File1 by the close method(). It avoids the reclosing of an already closed file by a subsequent call of a destructor. For example, the value of the file1Handle can be set by the call of a destructor, such as -1 and the checking can be done at the beginning for ensuring that the file1Handle is already equal to value (-1):

```
class File1 {
public:
    void close();
    ~File1();
private:
    int file1Handle_; // file1Handle_ >= 0 if/only-if
                      // it's open
};
File1::~~File1()
{
    close();
}
void File1::close()
{
    if (file1Handle_ >= 0) {
        //insert code to call the OS to close the file1...
        file1Handle_ = -1;
    }
}
```

Note that the other file methods may also need to check if the `file1Handle_` is -1 (i.e., check if the `File1` is closed).

Also note that any constructors that do not actually open a file should set `file1Handle_` to -1.

22. What are shallow and deep copies?

- A shallow copy is used to copy actual values of the data. It means, if the pointer points to dynamically allocated memory, the new object's pointer in the copy still points to items inside the old objects and the returned object will be left pointing to items that are no longer in scope. A copy of the dynamically allocated objects is created with the help of a deep copy. This is done with the help of an assignment operator, which needs to be overloaded by the copy constructor.

23. What do you understand by zombie objects in C++?

- In a situation, where an object is created in a class, a constructor fails before its full execution. It is very difficult to ensure about the execution of the constructor whether the constructor would return a value or not. Objects that are no

12 C++ INTERVIEW QUESTIONS

more required for an application are called zombie objects. These zombie objects occupy a space in the memory and wait for their termination.

24. Can a destructor be explicitly called if an object is created using the new operator?

- No, a destructor cannot be explicitly called if an object is created using the new operator. When you allocate the memory with the new operator, you are responsible for managing it yourself. To avoid cluttering up your program with memory that you are no longer using, you should delete it with the delete operator. The new operator returns a pointer to the newly allocated memory. Therefore, deleting the object is more preferable than to call the destructor explicitly. For example, if you allocate the object via a typical new expression, such as:

```
Jerry* j = new Jerry();
```

Then, the destructor `Jerry::~~Jerry ()` will automatically get called on deleting it via:

```
delete j; // Automatically calls j->~Jerry()
```

An explicit calling of the destructor would not release the memory that has been allocated for the Jerry object itself. The following are the two advantages of using the delete operator:

- ☐ It calls the destructor
- ☐ It deal locates the memory

25. Explain the use of the placement new method.

- The placement new method is used to place an object at a particular location in memory. For placing an object to its particular location, it allocates a pointer parameter to a new part of a new expression.

```
#include <new> // Must #include this to use "placement new"
#include <iostream>
```

```

class Tom
{
public:
void abc ()
{
    char memory[sizeof(Tom)];    // Line #1
    void* place = memory;        // Line #2

    Tom* j = new(place)
    Tom();    // Line #3 (see "DANGER" below)
    // The pointers j and place will be equal
}
};

int main()
{
    cout << "Enter the memory size: ";
    int size=20;
    cin>>size;
    int*Tom = new int[size];
    for (int i = 0; i < size; i++)
        Tom[i] = i;
    for (int i = 0; i < size; i++)
        cout << '\n' << Tom[i];
    delete [] Tom;
    return 0;
}

```

Line #1 creates an array of size of (Tom) bytes of memory, which is big enough to hold a Tom object. Line #2 creates a pointer place and points to the first byte of this memory. Line #3 essentially calls the constructor Tom::Tom (). The this pointer in the Tom constructor will be equal to place. The returned pointer j will therefore be equal to place.

Note: Use this placement new syntax only when an object is placed at a particular location in memory. For example, when you want to place an alarm object at a particular memory location and your hardware has a I/O timer device with a memory mapped.

The sole responsibility of a programmer is to destruct the placed object. To do this, the explicit calling of the destructor is needed:

```

void mno()
{
    char memory[sizeof(Tom)];
    void* b = memory;
    Bred* j = new(b) Tom();
    j
}

```

14 C++ INTERVIEW QUESTIONS

```
>~Tom(); // Explicitly call the destructor for the placed object
}
```

In the rare scenario like the preceding one, you need to explicitly call the destructor.

Note: This is a much clearer and more suitable way of handling the destruction / deletion situation.

26. On implementing a destructor, is there any need for the explicit calling of the destructors for member objects?

- No, there is no need to call the destructor explicitly. A destructor of a class, whether you explicitly define one or not, invokes a destructor automatically for its member objects. Their destruction is done in the reverse order of their declaration in the class.

Example:

```
#include <iostream.h>
#include <conio.h>
class Student
{
public:
~Student();
};
class Car{
public:
~Car();
private:
Student p_;
Student q_;
Student r_;
};
Student::~~ Student()
{
// Compiler automatically calls r_.~ Student()
// Compiler automatically calls q_.~ Student()
// Compiler automatically calls p_.~ Student()
}
Student obj1;
int main()
{
cout << "main () begins" << endl;
Student obj2;
```

```
{
cout << "r_begins" << endl;
Student obj3;
{
cout << "p_ends" << endl;
}
cout << "main() ends" << endl;
getch();
return 0;
}
```

27. On implementing a destructor of the derived class, is there any need for the explicit calling of the destructor for my base class?

- No, there is no need to call the destructor explicitly except in the case of placement new. A destructor of a class, whether you explicitly define one or not, invokes a destructor automatically for base class sub objects. Base classes are destructed after member objects. In the multiple inheritance, the destruction of the direct base classes occur in the reverse order of their appearance in the inheritance list.

Example:

```
#include <iostream.h>
#include <conio.h>
class Automobile
{
public:
Automobile(){}
virtual ~Automobile()
{
cout << "Executing Automobile dtor" << endl;
}
};
class Bus : public Automobile
{
public:
Bus(){}
~Bus()
{
cout << "Executing Automobile dtor" << endl;
}
};
int main()
```

16 C++ INTERVIEW QUESTIONS

```
{
Automobile* pAutomobile = new Bus;
delete pAutomobile;
getch();
return 0;
}
```

Note: Only the Automobile destructor message is displayed on the console window. Even though the object is Bus, the behavior of the delete operator remains unknown at runtime in relation to the type of the object pointed by the pAutomobile pointer.

28. How does List r; differs from List r();?

- Suppose that Member is the name of some class. Then, function f() declares a local member object called r:

Example:

```
void f()
{
List r; // Local object named r (of class Member)
}
//But function g() declares a function called r()
that returns a Member:
void g()
{
Member r(); // Function named r(that returns a
Member)
}
```

In the preceding code snippet, r is declared as a local variable and the other r() is declared as a function. That is the main difference.

29. Can one constructor of a class be called by another constructor of the same class for initializing the this object?

- No, not possible. Let's understand it with the help of an example.

Take a human class. Now, we want our constructor Human::Human(char) to call another constructor of the same class, say Human::Human(char, int), in order that

Human::Human(char, int) would help to initialize the this object. Unfortunately, there is no way to do this in C++.

It always occur in its specific form and not the way a user wants. For example, the line Human(h, 0); does not call Human::Human(char, int) on the this object. Instead it calls Human::Human(char, int) to initialize a temporary, local object (not this), then it immediately destructs that temporary object when control flows over the ;.

Example:

```
class Human {
public:
    Human(char h);
    Human(char h, int g);
};

Human::Human(char h)
{
    Human(h, 0); // this line does NOT help
    initializethe this object!!
}

//You can sometimes combine two constructors via a
//default parameter:
class Human {
public:
    Human(char h, int g=0); // this line combines the
    two constructors
};
```

If there is no default parameter for combining the two constructors, then there is a need of sharing of their common code in a private init() member function:

Another example:

```
class Human {
public:
    Human(char h);
    Human(char h, int g);
private:
    void init(char h, int g);
};

Human::Human(char h)
{
```

18 C++ INTERVIEW QUESTIONS

```
init(h, int(h) + 8);
}
Human::Human(char h, int g)
{
    init(h, g);
}
void Human::init(char h, int g)
{
    cout<< h <<endl;
    cout<< g << endl;
}
int main()
{
    Human h('s');
    getch();
}
```

30. Is the default constructor for a class Bred always Bred::Bred()?

- No, not always. A default constructor is a constructor which takes no arguments, and it exists so that you can create objects of your new class. An example of this is a constructor that takes no parameters:

```
class Bred {
public:
    Bred(); // Default constructor: can be called with
           no args
};
```

Another example of a default constructor is one that can take arguments, provided they are given default values:

Example:

```
class Bred {
public:
    Bred(int i=3, int j=5); // Default constructor: can
                           be called with args
};
```

31. On creating an array of bird_class objects, which constructor will get called?

- The default constructor will get called for the bird_class.

Example:

```
#include <iostream.h>
#include <string.h>
#include <conio.h>
class bird_class
{
public:
string name;
int number_seen;
void increment_count(){number_seen++;}
bird_class(string text);
bird_class();
};
bird_class::bird_class(string text)
{
name = text;
number_seen = 0;
}
bird_class::bird_class()
{
name = "Robin";
number_seen = 0;
}
int main()
{
bird_class orioles;
orioles.name = "oriole";
orioles.increment_count();
cout <<"Numbers of " << orioles.name << "s seen:"
<< orioles.number_seen << endl;
getch();
return 0;
}
```

If a normal constructor is provided, the compiler will not create a default constructor. A constructor can be simply created with no arguments, as described in the preceding example.

32. Can we use initialization lists or assignments in a constructor?

- The constructor uses initialization lists. In fact, all the member objects in the initialization list should be initialized by a constructor. Let's take an example of One exception.

Consider the following constructor that initializes member **object r_** using an initialization list: **Harry::Harry() : r_(whatever) { }**. This results in the improved performance. For example, if a member **variable r_** is of the same type as the **whatever** expression, then the compiler do not have to make a separate copy of the object and the outcome related to the **whatever** expression is constructed directly inside **r_**.

Even if the types are not same, the compiler is able to initialize objects with initialization lists, even if the types are different than assignments.

Assignment is another way of building constructors, such as, **Jimmy::Jimmy() { j_ =whatever; }**. In this case, the temporary and separate objects are created as a result of the expression **whatever** and the temporary object is passed into the **j_** object's assignment operator.

By the use of an assignment operator in a constructor, the default constructor has fully constructed the member object and results in the allocation of some amount of memory by default or the opening of some default file. The major drawback of using the assignment operator is that all the work done may be lost if the **whatever** expression or the object of the assignment operator closes the particular file or release that memory. For example, if a wrong file is opened.

Conclusion: Initialization list is better and faster for running the code rather than assignment operators.

Note: The initialization of objects is more symmetrical in initializing data members in the initialization lists and also favors the built-in/intrinsic types constructors, such as non-static constructors and non-static reference data members which cannot be assigned with a value by the assignment operator. So, for making the symmetry it is better to initialize all the variable and data members in the initialization list.

33. Should the **this** pointer can be used in the constructor?

- We can use the **this** pointer in the constructor in the initialization list and also in the body. However, there is a feeling that the object is not fully formed so we should not use **this** pointer. Let's understand the use of the **this** pointer with the help of the following example. The declared data members of a base class and/or the declared data members of the constructor's own class can be accessed by the constructor {body} and/or a function called from the constructor. This is possible because of the full construction of constructor's body at the time of execution. The preceding example always works.

The override in the derived class is not possible for a function called from the constructor and the {body} of a constructor which is independent of calling the virtual member function by the explicit use of the **this** pointer.

Please make sure that the initialization of other data member has already been done before passing the data member to another data member's initializer in **this** object. With the help of some straightforward language rules (independent of the specific compiler that you are using), you can confirm about the determination of initialization of data members.

Without the prior knowledge of these rules, there is no use of passing any data member from the **this** object.

34. What do you mean by the term named constructor?

- Named constructor is a technique that is used to create private constructors. It also creates static functions for doing

the basic and safer construction operations for users of a class. The constructor always has the same name as that of the class. Therefore, while using multiple constructors, you must use a different parameter list for each constructor to differentiate between them. In the named constructor, the declaration of the class's constructor in the private or protected sections and declaration of public static methods for returning an object becomes possible. These static methods are also called as Named Constructors. There is a common static method available for constructing an object in different ways.

For example, suppose we are building the Shape class that represents a position on the L-B plane. The two ways to specify a 2-space coordinate are rectangular coordinates (L*B), parallelogram coordinates (Breadth*Height). Note that the two parameters of data type for these two coordinate systems are the same which means the double data type is used for both rectangular and parallelogram. It results in a situation that is prone to errors in the overloaded constructors:

Example:

```
#include <iostream.h>
#include <conio.h>
class Shape
{
public:
    Shape(double l, double b); // Rectangular
    coordinates
    Shape(double b, double h); // Parallelogram
    coordinates (breadth and height)
    // ERROR: Overload is Ambiguous
    Shape::Shape(double, double);
};

int main()
{
    Shape p = Shape(5.2, 2.3); // Ambiguous: which
```

```
coordinate system?
}

//One way to solve this ambiguity is to use the
Named Constructor:
#include <iostream>
#include <math.h>
#include <conio.h>
using namespace std;

class Shape
{
public:
    static Shape Rectangle(double l, double b);
    static Shape Parallelogram (double b, double
h);
private:
    Shape (double l, double b);
    double l_,b_;
};

    inline Shape::Shape(double l, double b)
: l_(l), b_(b) {
    cout<<l*b<<endl;
}

inline Shape Shape :: Rectangle (double l, double
b)
{
    cout<<"Area of Rectangle: " << endl;
    return Shape (l,b);
}

inline Shape Shape :: Parallelogram (double b,
double h)
{
    cout<<"Area of Parallelogram: " << endl;

    return Shape (b,h);
}

int main()
{
```

24 C++ INTERVIEW QUESTIONS

```
Shape p1=Shape::Rectangle (5.2,2.3);  
Shape p2=Shape::Parallelogram (5.7,3.2);  
getch();  
return 0;  
}
```

If you want the Shape class to have derived classes, then you need to make sure that your constructors are in the protected section. With the use of the Named Constructor Idiom, the two types of shapes are declared as static in public. The returning of values of shape of rectangle and parallelogram is possible because of the declaration of class's constructor in the private section.

2

Virtual Functions

1. What is a virtual function?

- In C++, virtual function is defined as a member function in a class from which other classes may be derived. If the derived class has a function with the same name and parameter list, the derived class's function is always executed for objects of the derived class. We have to use virtual class in the base class for defining this function in the base class. Let's understand this concept with the help of the following example:

```
#include <iostream.h>
#include <conio.h>
class A
{
public:
char Avar;
void Show()
{
cout << "i'm A" << endl;
}
};
class B : virtual public A
{
public:
void show()
{
cout << "i'm B" << endl;
}
};
class C : virtual public A
{
public:
void show()
{
cout << "i'm C" << endl;
}
};
class D : public B, public C
```

```

{
public:
void Show()
{
cout <<"i'm D" << endl;
}
};
int main()
{
D Dtemp;
Dtemp.Show();
getch();
return 0;
}

```

In the preceding example, the classes B and C use the virtual keyword to inherit the members of class A. Therefore, class A is the virtual base class and class D inherits only one copy of the member data Avar from class A. Therefore, any reference to the member Avar through an object of class D does not lead to ambiguity.

2. Define pure virtual function.

- Pure virtual function is defined as a virtual function in a base class. It is implemented in a derived class. A program may not declare an instance of a class that has a pure virtual function. A program may not declare an instance of a derived class if that derived class has not provided an overriding function for each pure virtual function in the base.

A pure virtual function can be created through the following way:

```

class Car
{
void jar() = 0;
}
Car MyCar; // compilation error

```

3. What do you understand by a pure virtual member function?

- A virtual function with no body structure is called a pure virtual member function. You can declare a pure virtual member function by adding the notation =0 to the virtual member function declaration. For example, area () is a virtual function, but when it is declared as shown in the following expression, it becomes a pure virtual member function:

```
virtual int area () = 0;
```

To avoid a compilation error, all the classes need to implement pure virtual classes that are derived from the abstract class.

4. How can virtual functions in C++ be implemented?

- When a class has at least one virtual function, the compiler builds a table of virtual function pointers for that class. This table, commonly called the v-table, contains an entry for each virtual function in the class. The constructor of the class is used to create this table. Each object of the class in memory includes a variable called the `vp`, which points to the class's common `vtbl`. The `_rst` (which creates the `vtable`) is constructed by the base classes after the construction of the derived classes. The derived class constructor overwrites the entries of the `vtable`, if its constructor overrides any virtual function of the base class. Therefore, you need not call every function from a constructor just to avoid the error prone scenario of calling the

Base class constructor instead of the derived class constructor (which fails to set the entries of objects of `vtable`).

5. What do you understand by pure virtual function? Write about its use?

- A pure virtual function in a base class must have a matching function in a derived class. A program may not declare an instance of a class that has a pure virtual function. A program may not declare an instance of a derived class if that derived class has not provided an overriding function for each pure virtual function in the base. If you want programmers to override certain functions in a class, such as those that need information customized for a particular installation, you should make these functions pure virtual.

6. How the virtual functions maintain the call up?

- The call is maintained through `vtable`. Each object includes a variable known as `vp`, which points to the class's common `vtable`. It contains an entry for every virtual function.

On calling the function, vtable calls the appropriate function using vptr.

7. Write a note on virtual destructor.

- As you know, the destructor of the derived class is not invoked when you pass a derived class object to a base class pointer. In this situation, by making the destructor of base class as virtual, you can invoke the destructors of both the base and derived classes, in the order, derived- to- base classes.

Listing1 shows how to code a virtual destructor in a base class.

Listing 1: Creating a Virtual Destructor in a Base Class

```
#include <iostream.h>
#include <conio.h>
class Base
{
    public:
        Base()
        {
            cout<<"Constructor: Base"<<endl;}
        virtual ~Base(){ cout<<"Destructor :
Base"<<endl;} //virtual destructor
};
class Derived: public Base
{
    public:
        Derived(){ cout<<"Constructor:
Derived"<<endl;}
        ~Derived(){ cout<<"Destructor :
Derived"<<endl;}
};
void main()
{
    Base *b = new Derived ();
    delete b;
    getch();
}
Output:
Constructor: Base
Constructor: Derived
Destructor : Derived
Destructor : Base
```

You can declare a virtual destructor with an empty body, as shown in the following code snippet:

```
virtual ~Base(){}

```

3

Inheritance

1. Write a note about inheritance.

- Inheritance is a mechanism that allows the object of one class to utilize the form and model or behavior of another class. A class derived from the base class inherits attributes, functions, or methods that implement the base class to the derived class and also extends the derived class by overriding functions and adding new additional attributes and functionalities.

2. What is the need of multiple inheritance?

- The multiple inheritance allows you to define a new class that inherits the characteristics of several base classes which are not related to each other. The vehicle class encapsulates the data and behavior that describe vehicles along with other information, such as date of purchase, life, and maintenance schedules. Classes that support specific kinds of vehicles, such as trucks, airplanes, and cars are also derived from the vehicle class. The asset class encapsulates the data and behavior of the organization's assets, including acquiring date, and depreciation schedule data for accounting purposes. A company car, being both a vehicle and an asset, is represented by a class that derives from both base classes.

Now, the question arises that why the vehicles, asset data, and behavior are not included in the CompanyCar class? The reason is two-fold. First, not all vehicles are assets. The company might also hire cars on lease. Now, these hired

vehicles cannot be considered as assets in the accounting sense. Second, not all assets are vehicles. The company may have many other assets, such as office furniture, copiers, and computers.

3. Write about the IS A and HAS A class relationships and also explain about their implementation in a class.

- With the help of object-oriented designers, inheritance is used to define relationships, such as the derived class *is a* kind of the base class.

A car *is a kind of* vehicle. An engineer is a kind of employee, which *is a kind of* person. This relationship is called the *IS A* relationship.

Inheritance is not appropriate for relationships where one class has an instance of another. For example, an employee *has a* date of birth and a department *has a* manager. This relationship is called the *HAS A* relationship. The *HAS A* relationship is appropriate when the embedded object is singular and belongs to the class that embeds it. An employee's date of birth belongs to that employee. Even when another employee has the same birth-day, each one has a private copy of the data element.

4. Explain the concept of template.

- Templates allow you to describe a generic type to manage other data types. Using templates, you can implement the algorithm once in your code and use it with many different types of data, even with objects. Templates are of two types—Class template (which is used to build general-purpose container classes, such as stacks, lists, and queues) and Function template (which is defined as a function using an abstract classes, non-specific data types and then use that template to create functions that use specific data types, such

as int or float). The template specifier has the following two parameter types:

- **template <class identifier>** function_declaration
- **template <typename identifier>** function_declaration

5. **How can you say that a template is better than a base class?**

- In case of designing a generic data type to manage objects of other data types which are unknown to their container class, a template is preferred over a base class. A template can contain more than one data type parameter, making it possible to build parameterized data types of considerable complexity. The class template also allows the declaration of more than one object in the same program.

6. **Explain the concept of multiple inheritance (virtual inheritance). Write about its advantages and disadvantages.**

- Multiple inheritance is defined as the ability of a derived class to inherit the characteristics of more than one base class. For example, you might have a class zoo which inherits various animal classes. The advantage of multiple inheritance is that you can create a single derived class from several base classes. It helps in understanding the logic behind these complex relationships.

The multiple inheritance is not preferred by the programmers because with multiple base classes, there are chances of conflicts while calling a method in the sub class that exists in both the base classes. For example, if you have a class named **zoo** that inherits both the **Anaconda** and **Zebra** classes, then each base class might define a different version of the method **walk**, which creates a conflicting situation.

7. Describe inheritance and non-inheritance of a derived class.

- Using inheritance, you can derive all the data members and all the other common functions of the base class and still retain the functionality of the common methods of the base class. For example, the **animal** class with **eat**, **sleep** and **breathe** methods comprises the base class because they are common to all animals. Now, a new class **Elephant** with a method **trumpet** is derived from the class **animal**. With the help of inheritance, the **Elephant** class inherits the **trumpet** method and still retains the **eat**, **sleep**, and **breathe** methods from the base class, **animal**.

Therefore, you can add new methods to an existing class with the retaining of common methods. There are few methods that are not inherited on creating a derived class which includes constructors, destructors, and assignment operator methods of their base classes.

8. Write a note about the virtual member function.

- A virtual function is a member function of the base class and relies on a specific object to determine which implementation of the function is called. However, a virtual function can be declared a friend of another class. If a function is declared virtual in a base class, you can still access it directly using the `::` operator. Note that if you do not override a virtual member function in a derived class, a call to that function uses the function implementation defined in the base class.

9. How static typing is achieved by dynamic binding in C++?

- Pointers to base class objects can also be used to point to derived class objects (although you cannot reference members in the derived class that are not in the base class when you use a base class pointer). As one pointer can point to various derived objects, this process is called

polymorphism. For example, a `fruit*` class that actually points to an Apple object. There are two types of pointers:

1. Static pointer means the actual type of pointer of the base class; `fruit`, in this case
2. Dynamic pointer means the type of the object to be pointed; `apple`, in this case

Static typing ensures that the execution at run-time will only get values that are of the right type for the program. This typing detects error in the early stages of a program, minimizing the possibility of error occurrence at the run time. This helps in improving the performance of the program.

Dynamic binding ensures that the function calls are resolved at run time; thereby, providing users the flexibility to alter a call without having to modify the code. For a programmer, efficiency and performance would probably be a primary concern but to a user, flexibility and maintainability are much more important. Dynamic binding represents dynamic polymorphism. Dynamic binding is based on overriding principles that are based on inheritance. The dynamic binding is used when a function call is matched with the correct function definition at run time. You can declare a function with the `virtual` keyword, if you want the compiler to use dynamic binding for the specific function.

10. How calling of virtual member functions differ from that of non-virtual member functions?

- The calling of the non-virtual member function depends on the selection of a member function at the compile time based on the type of a pointer or reference to the object. However, you can make the program wait until it actually runs before deciding what version of a particular member function to call, which is called late or dynamic binding. You declare a function with the `virtual` keyword if you want the compiler to use dynamic binding for the specific function.

When a class has at least one virtual function, the compiler builds a table of virtual function pointers for that class. This table, commonly called the v-table or the virtual-table, contains an entry for each virtual function in the class. Each object of the class in memory includes a variable called the v-pointer, which points to the class's common virtual table. For example, the Button class has virtual functions for drag() , drop() and click(). There is one virtual table that is associated with the class Button. This virtual table allows the v-pointer of each of those button objects to point to objects related to the v-table only. There is a separate v-pointer to each of the virtual functions of a class. For example, the Button v-table with three pointers are:

```
a pointer to Button::drag (), a pointer to
Button::drop(), and a pointer to Button::click().
```

During the execution of a virtual function, the program accesses the object's v-pointer related to the v-table of a particular class.

11. Write about the performance cost of using virtual functions in C++.

- The performance cost of virtual functions means that when a class has at least one virtual function, the compiler builds a table of virtual function pointers for that class. This table commonly called the vtable contains an entry for each virtual function in the class. A vptr exists as a variable within each object of the class in memory. A common vtable of a class is pointed by the vptr. A vptr exists as a hidden member of the object created. A vptr helps to initialize an object of a particular class with the help of its constructor.

Let's understand this with the help of an example:

```
#include <iostream.h>
#include <conio.h>
class River
{
public:
```

```

        virtual void wave1() {};
        virtual void wave2() {};
};
class S1: public River
{
public:
    virtual void wave1() {};
};
class S2: public River
{
public:
    virtual void wave2() {};
};
int main()
{
    cout << " wave 1" << endl;
    getch();
    return 0;
}

```

In the preceding code snippet, there are three classes. Therefore, the setting of 3 virtual tables for each of these classes (River, S1, S2) is done by the compiler.

The creation and addition of the virtual pointer is also done by the compiler, as shown in the following code snippet:

```

#include <iostream.h>
#include <conio.h>
class River
{
public:
    FunctionPointer *__vptr;
    virtual void wave1() {};
    virtual void wave2() {};
};
class S1: public River
{
public:
    virtual void wave1() {};
};
class S2: public River
{
public:
    virtual void wave2() {};
    getch();
};

```

The compiler sets the vptr in the similar way as that of virtual functions. It means a virtual pointer points to the virtual table of the particular class after creating an object. For example, when an object of the River class is created, the vptr point to the v-table for the River class with the help of the compiler.

12. Write about the functional overriding of the base class by the member function in the derived class.

- You can customize a base class by redefining its members in the derived class. This process is called overriding. When you override a base class's member, you redefine the member, whether it is a data member or a method. For example, the animal class has a method named breathe that prints "Breathing...", but if you derive a class named fish from the animal class, you might think it more appropriate if that method were to print out "Bubbling...". To override the base class's method, you simply need to redefine that method. Let's understand this concept with the help of the following code snippet:

```
#include <iostream.h>
#include <conio.h>
class animal
{
public:
void eat();
void sleep();
void breathe();
};
class fish : public animal
{
public:
void breathe();
};
void animal :: eat(){cout << "Eating..." << endl;}
void animal :: sleep(){cout << "Sleeping..." <<
endl;}
void animal :: breathe(){cout << "Breathing..." <<
endl;}

int main()
{
```

```

    animal fish;
    fish.breathe();
    getch();
    return 0;
}

```

Note that the overridden breathe method displays "Breathing". As you can see, the fish class's breathe method has overridden the base class's breathe method. In this way, you can still use a base class, even if a few methods or data members are inappropriate by redefining that particular method in a derived class.

13. Explain the concept of dynamic binding with the help of an example.

- The matching of a function call by the C++ compiler with the correct function definition at run time is called dynamic binding. However, to implement dynamic binding, the concept of virtual functions is important.

Let's understand the concept of dynamic binding with the help of the following example-

```

#include <iostream.h>
#include <conio.h>
class animal
{
public:
    void sleep();
    void breathe();
    void eat();
};
class fish : public animal
{
public:
    void eat();
};
void animal::eat(){cout << "eating..." << endl;}
void animal::sleep(){cout << "sleeping..." << endl;}
void animal::breathe(){cout << "breathing..." << endl;}
void fish::eat(){cout << "opening mouth..." << endl;}

```

```
int main()
{
    animal animal_object;
    fish bigfish;
    animal *pointer;
    pointer = &animal_object;
    pointer->eat();
    pointer = &bigfish;
    pointer->eat();
    getch();
    return 0;
}
```

In the preceding code snippet, if a pointer holds a pointer to an animal class object and you use it to call **eat**, **animal::eat** will be called. However, if a pointer holds a pointer to a **fish** class object and you use it to call **eat**, **fish::eat** will be called, not **animal::eat**. Using late binding, the program determines about the members of a class to which a pointer points at the runtime, not at the compile time.

14. When should a destructor be made virtual?

- You need to make the destructor virtual when an object of a derived class is declared with the **new** operator and that the pointer type is that of a base class with a non-virtual destructor. When the program destroys the object by using the **delete** operator with the pointer as an argument, the base destructor executes instead of the derived destructor.

If the base class needs no custom destruction, you must still provide a virtual destructor (with an empty statement block) to permit the appropriate call for the destructor in relation to the dynamically allocated objects. All the classes in the personnel class hierarchy have virtual destructors.

15. Should the member functions which are made public in the base class be hidden?

- As the public member functions of the base class are necessary to implement the public interface of the class, the

member functions which are made public in the base class should never be hidden. When you are designing a class, make the derived data members private because a private member of the derived class is accessed through the protected or public member functions of the base class.

16. Write a note about upcasting.

- Upcasting casts a pointer or a reference from a derived class to one of the base classes in the same hierarchy. Most upcasting is done by implicit conversion. The compiler performs a static type-check to ensure that the conversion from the derived class to the base class through a pointer is allowed, and if it is not allowed, a compile-time error message appears. You use `dynamic_cast` for upcasting when you need to determine at runtime whether a pointer to a derived class really contains the address of an object of that class and at the same time, you can force the address into a pointer of one of the object's base classes.

17. Can a derived array be called a kind-of-base array?

- A derived array cannot be called a kind-of-base array. Let's understand this concept with the help of the following code snippet:

```
class Parent {
public:
    virtual void a();           // 1
};

class Child : public Parent {
public:
    ...
private:
    int n_;                     // 2
};

void ABC(Parent* arrayOfParent)
{
    arrayOfParent[1].a();       // 3
}
```

```

    }

    int main()
    {
        Derived arrayOfChild[12];    // 4
        ABC (arrayOfChild);          // 5
        ...
    }

```

In the preceding code snippet, the pointer declared to the derived class **arrayOfChild** has large size as compared to the pointer declared to the **base class arrayOfParent** class. The **pointer arithmetic** (which is used to keep adding 1 to a pointer to move from one value to the next) and the **sizeof(Base)** is used by the compiler internally when estimating the address for **arrayOfParent[1]**. Line 3 is incorrect because the **sizeof** operator is specific in terms of giving the size of array and pointer. The preceding code snippet will produce error-prone conditions.

The main problem of C++ is that it failed to differentiate between a pointer –to –an-array and a pointer-to-a-thing.

18. Can an array be replaced with a template container class?

- Arrays are very similar to pointers. You cannot add values to an array as well as to a pointer. Instead of using arrays, a template container class, such as vector class from Standard Template Library (STL) should be used. You can add information to and remove information from the end of the vector class without much overhead. Instead of using raw arrays, using namespace `std::vector<T>` should be used.

19. Can circle be called an ellipse?

- Yes, a circle can be called an ellipse. Let's understand this concept with the help of an example, if ellipse has a member function named as `setsize` with the `width()` of the object as `x` and its `height()` as `y`. There are two kinds of relationships that exist between a circle and an ellipse:

- *Circle and ellipse can be made as two different classes —*
In this case, ellipse can be derived from AsymmetricShape class and has a member function named as setSize(x,y). On the contrary, circle can be derived from SymmetricalShape class and has a member function named as setSize(size). Therefore, they both are unrelated in its member functions as well as in their derivations.
- *Circle and ellipse can be derived from a base class —*In this case, circle and ellipse both can be inherited from the class Oval because class Oval can only have setSize(size) member function which sets the height() and width() to the size of the object.

20. Why do we separate interface from implementation?

- The interface is visible to the user of the class and consists of public members, which are usually member functions. The class user reads and modifies values in data representation by calling public member functions. The interface is generic in that it is not bound to any particular implementation.

The implementation of a class, which consists of private data members and private member functions, is essentially hidden from the program. The implementation defines the details of how the class implements the behavior of the abstract base type. The class author should be able to change the implementation without affecting the program.

21. How can you differentiate between inheritance and implementation in C++?

- With the help of abstract base classes, we can differentiate between interface and implementation in C++.

22. Write about abstract base classes.

- An abstract base class is a class definition that is always a base class for other classes to be derived from. No specific objects of the base class are declared by the program. A C++ abstract base class is one that has a pure virtual function, a protected constructor, or a protected destructor.

At the design level, you create a pure virtual method by using `= 0` in place of a method body. This notation specifies that the member function is a pure virtual function. This means that the base class is an abstract base class, and the class designer intends the class to be used only as a base class. The base class may or may not provide a function body for the pure virtual function. In either case, a program that uses this class may not directly declare any objects of the abstract base class. If the program declares an object of a class directly or indirectly derived from the abstract base class, the pure virtual function must be overridden explicitly.

23. Write about pure virtual member function.

- You can leave a base class method unimplemented if you make it a pure virtual method. A class's member function declaration includes an equal sign followed by a zero (`=0`). This means that the base class is an abstract base class and the class designer intends the class to be used only as a base class. The base class may or may not provide a function body for the pure virtual function. In either case, a program that uses this class may not directly declare any objects of an abstract base class. If the program declares an object of a class directly or indirectly derived from the abstract base class, the pure virtual function must be overridden explicitly.

Let's take an example. There is an animal class and you need to make the breathe function in the animal class as a pure virtual function:

```
#include <iostream.h>
using namespace std;
class animal
{
public:
virtual void breathe ( ) = 0;    // 0 means it is
    "pure virtual"
};
```

Now, the **animal** class is an abstract class and you cannot create objects of this class. You can only use it as a base class. To create a non-abstract derived class from this class, you must override the **breathe** function, as follows:

```
#include <iostream.h>
#include <conio.h>
class animal
{
public:
virtual void breathe ( ) = 0;
};
class fish: public animal
{
public:
void breathe ( );
};
void fish::breathe( )
{
cout << "Bubbling..." << endl;
}
int main( )
{
    fish bigfish;
    bigfish.breathe();
    getch();
    return 0;
}
```

In the preceding code snippet, the **animal** and **fish** classes each have a different version of a function named **breathe**. That means that **pointer->breathe** calls **animal::breathe** when pointer points to an **animal** object, but it calls **fish::breathe** when pointer points to a **fish** object.

24. Explain about a copy constructor or an assignment operator's declaration which contains a pointer to a (abstract) base class?

- A virtual constructor is used to declare an object of a copy constructor or an assignment operator in a class containing an abstract base class pointer. In this method, a pure virtual clone() method is declared in the base class, as shown in the following code snippet:

```
#include <iostream>
class Herbivorous
{
public:
    virtual Herbivorous* clone() const = 0;    // T
    e Virtual (Copy) Constructor
};
```

Then, the clone() method in each derived class is implemented. The following code snippet shows the code for the derived class Elephant:

```
class Elephant : public Herbivorous
{
public:
    virtual Elephant* clone() const;
};
Elephant* Elephant::clone() const
{
    return new Elephant(*this);
}
```

Note: The return type of the derived and base class is made different intentionally.

The following code snippet shows the code for the derived class Cow:

```
class Cow : public Herbivorous {
public:

    virtual Cow* clone() const;
};
Cow* Cow::clone() const
{
    return new Cow(*this);
}
```

Now, at runtime, the behavior of an object of the class `Abc` is unknown in relation of a call to the `Elephant` or `Cow`. So, for copying the object, the `Abc`'s copy constructor and assignment operator will invoke the `clone` method of the `Herbivorous` class:

```
class Abc
{
public:
    // p must be a pointer returned by new; it must
    // not be NULL
    Abc(Herbivorous* p)
        : p_(p) { (p != NULL); }
    ~Abc ()
        { delete p_; }
    Abc(Abc const& f)
        : p_(f.p_>clone()) { }
    Abc& operator= (Abc const& f)
        {
            if (this != &f)
            {
                // Check for self-assignment
                delete p_;
                // ...THEN delete the old one
                p_ = 0;
            }
            return *this;
        }
private:
    Herbivorous* p_;
};

int main()
{
    char clone;
    cout << "invoke the clone method of herbivorous
class" << endl;
    getch();
    return 0;
}
```

25. Write about private inheritance.

- When you use private inheritance, the public and protected members of the base class become private members of the

derived class. In fact, private inheritance implements a has-a relationship more than a is-a relationship.

26. Write about different relationships that exist with the private inheritance.

- Let's understand this concept with the help of the following example: Use of the access specifier as public when specifying how to derive a new class from a base class:

```
class elephant : public animal
{
public:
    void trumpet();
    void stampede();
};
```

Here, the private members of animal are not accessible, but the public members of animal is accessible.

When you use private inheritance, the public and protected members of the base class become private members of the derived class.

In fact, you can think of private inheritance as implementing a has-a relationship more than a is-a relationship, because the base class is far more contained than in public inheritance.

Here's an example. In the following code snippet, you derive the **elephant** class from the **animal** class using private inheritance. Note that the methods of the **animal** class are no longer available from an object of the **elephant** class, because those methods are now private, so if you want to call one, you have to implement that call from code inside the **elephant** class, as with the **breathe** method:

```
#include <iostream.h>
#include <conio.h>

class animal
{
public:
    void breathe();
public:
```

```

void eat();
void sleep();
};
class elephant : private animal
{
public:
void trumpet();
void stampede();
void breathe();
};

void animal :: eat(){cout << "Eating..." << endl;}
void animal :: sleep(){cout << "Sleeping..." <<
endl;}
void animal :: breathe(){cout << "Breathing..." <<
endl;}

void elephant :: trumpet(){cout << "Trumpeting..." <<
endl;}
void elephant :: stampede() {cout << "Stampeding..."
<< endl;}
void elephant :: breathe() { animal :: breathe();}
int main()
{
elephant jumbo;

jumbo.breathe();
jumbo.trumpet();
jumbo.stampede();
getch();
return 0;
}

```

27. Which is more preferable between composition and private inheritance?

- Composition is more preferred as compared to private inheritance. Use of composition is preferred when needed and use of private inheritance is preferred in the specific conditions.

Private inheritance provides you with the assurance that the code should not be accessed by any other class from outside. However, it is suitable in conditions when you want your

public and protected members of the base class become private members of the derived class.

Different types of compositions mean class relationships, such as a has-a and is-a relationship. In a has-a relationship, instead of using inheritance, the class that has an object of another class embeds the object as a data member. Let's understand this with the help of the following code snippet:

```
#include <iostream.h>
#include <conio.h>
class legs
{
public:
void walk(){cout << "walking..." << endl;}
};
class animal
{
legs l;
public:
void eat();
void sleep();
void breathe();
void walk(){l.walk();}
};
void animal::eat(){cout << "Eating..." << endl;}
void animal::sleep(){cout << "Sleeping..." << endl;}
void animal::breathe(){cout << "Breathing..." <<
endl;}
int main()
{
animal jumbo;

jumbo.eat();
jumbo.breathe();
jumbo.sleep();
jumbo.walk();

getch();
return 0;
}
```

In the preceding code snippet, a **legs** class with one method, **walk** is created. Then an object of the legs class is embedded in the animal class. This allows the **animal** class to build a

has-a relationship with the **legs** class and add as many classes and members as required.

On the other hand, private inheritance is used when you want to call the derived methods of a base class from an object of the derived class. In this case, the implementation of a call is done inside the code of the derived class. Let's understand this concept with the help of the following code snippet:

```
#include <iostream>
#include <conio.h>
class animal
{
protected:
    void eat();
    void sleep();
};

class rabbit : private animal
{
public:
    void sleep();
    void eat();
};

void animal :: sleep(){cout << "sleeping..." << endl;}
void animal :: eat(){cout << "eating..." << endl;}
void rabbit :: eat(){animal::eat();}

int main()
{
    rabbit hop;
    hop.eat();
    getch();
    return 0;
};
```

In the preceding code snippet, you need to derive the **rabbit** class from the **animal** class using the private inheritance. Note that the methods of the **animal** class are no longer available from an object of the **rabbit** class, because those methods are now private, so if you want to call one, you have to implement that call from the code inside the **rabbit** class, as with the **eat** method.

28. Write about void pointers and its declaration.

- A void pointer can point to any kind of variable. You can use a pointer of the `void*` type to hold a pointer to any non-constant data type. In fact, void pointers are sometimes called generic pointers. It is declared as follows:

```
void* vptr;
```

Let's take an example in which you need to use a void pointer to point to an integer value:

```
#include <iostream>
using namespace std;
int main()
{
    int integer = 5;
    void *pointer;
    pointer = &integer;
    .
    .
    .
}
```

Now, you can access the value which is pointed by the this pointer. Use a cast method for indicating the type of pointer in relation to the void pointer. In this case, you want to treat the void pointer as an integer pointer:

```
#include <iostream>
#include <conio.h>
int main()
{
    int integer = 5;
    void *pointer;
    pointer = &integer;
    cout<<"The value in the integer is "
    << *(static_cast<int *> (pointer)) <<endl;
    getch();
    return 0;
}
```

29. Write about the relationship between the private and protected inheritance.

- When you derive a base class using protected access specifier, the private members of the base class become protected members of the derived class (the private members

of the base class are not available to the derived class). Protected inheritance is a good technique to use when you derive a class based on a class that was not written for inheritance because it converts public members of the base class, which you might not want to expose to the protected members of the derived class.

Note that in case of private inheritance, only members of a base class can access private members of that base class. While in the case of protected inheritance, both members of its base class as well as its derived class can access the protected members of that base class.

30. Which should be more useful: the protected and public virtuals?

- Public virtuals are more preferable than protected virtuals. Public virtuals permit a program to directly modify the values of data members. It can be accessible to any function that is within the scope of the structure. It translates the implementation for the user of the class. It is the most common, convenient, and easiest way to implement by any of the programs.

On the other hand, protected virtual functions are used for hiding some methods or data members from the outside class making it consistent and symmetrical in its approach. It is condition specific, so not commonly used.

31. Describe the setting up of my member functions to avoid overriding by the derived class.

- To avoid overriding of the member functions by the derived class, the leaf method is used. With the help of the leaf method, it is possible to leave the code unaligned at the time of execution by simply adding a comment next to the method. This method is easy, fast, and inexpensive to use.

32. Write about the rules of accessing the private and protected inheritance.

- Let's take an example-

```
class R { /*...*/ };
class S_priv : private R { /*...*/ };
class S_prot : protected R { /*...*/ };
class S_publ : public R { /*...*/ };
class Abc { R r; /*...*/ };
```

In the preceding code snippet, the private R cannot be accessed by the derived classes. The public and protected parts of R are made private in S_priv and made protected in S_prot. Similarly, the public parts of R are public in S_publ. Class Abc can only access the public parts of R. To make the members of class R as public in S_priv or S_prot, use R::prefix to state the member of R.

Let's understand this concept with the help of an example. To make member R::f(int, float) public in S_prot, the following code snippet is used :

```
class S_prot : protected R {
public:
    using R::f; // Note: Not using R::f(int, float)
};
```

33. Write a note on dreaded diamond.

- The appearing of a particular class more than once in a hierarchy of class inheritance is known as dreaded diamond. Let's understand this concept with the help of the following example:

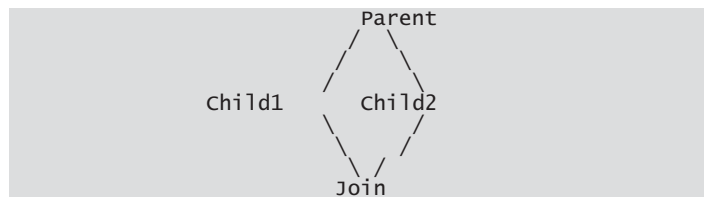
```
#include <iostream>
class Parent
{
public:
protected:
    int data_;
};
class Child1 : public Parent {};
class Child2 : public Parent {};
class Join : public Child1, public Child2 {
public:
```

```

void method()
{
    data_ = 1;  ← bad: this is ambiguous; see below
}
};
int main()
{
    Join* j = new Join();
    Parent* p = j;  ← bad: this is ambiguous; see below
}

```

The final look of hierarchy of inheritance is as follows:



The advantage is:

It defines any of the data members declared in the Parent class, such as `data_`, will appear twice within a Join object.

The Disadvantage is:

It enables the Conversion of `Join*` to `Parent*` or from `Join&` to `Parent&` can create error-prone situations. Many questions may arise. For example, which type of data you may want to change? Which sub object of the base class do you want?

Solutions of these error-prone situations: inform the compiler of C++ about the appearance of the Parent class sub object to appear only once within a Join object.

Another solution is: Make `Child2::data_ = 1` instead of `data_ = 1` or conversion of `Join*` to a `Child1` and then to a `Parent*` could be done.

34. Write about “delegate to a sister class” through virtual inheritance.

- Let's take an example:

```

#include <iostream.h>
#include <conio.h>

```

```
class Automobile
{
public:
    virtual void petrol() = 0;
    virtual void diesel() = 0;
};
class Car1 : public virtual Automobile {
public:
    virtual void petrol();
};
void Car1::petrol()
{ petrol(); }
class Car2 : public virtual Automobile {
public:
    virtual void diesel();
};
void Car2::diesel()
{ diesel(); }
class Join : public Car1, public Car2
{
public:
};
int main()
{
    Join* t1 = new Join();
    Car1* t2 = t1;
    Car2* t3 = t1;
    t1->petrol();
    t2->diesel();
    t3->petrol();
    getch();
    return 0;
}
```

In the preceding code snippet, on calling the `Car1::petrol()` calls `this->diesel()` class, the `Car2::diesel()` class is called. The unknown class means class `Car2::diesel()` overrides the virtual function invoked by the `Car1::petrol()` class, known as cross-delegation (the process of customization of the behavior of polymorphic classes).

4

Polymorphism

1. Explain polymorphism with the help of an example.

- Polymorphism is defined as the exhibition of a unique behavior or response by different objects when responding to the same message. Pointers to base class objects can also be used to point to derived class objects (although you cannot reference members in the derived class that are not in the base class when you use a base class pointer). For example, Teacher and Doctor classes derived from the base Professional class exhibit different behavior when a member function is called for one of their objects through a pointer or reference to the base Professional class. The Doctor class in our example assumes the existence of only one kind of doctor. In reality, a Professional class can employ doctors from various streams: cardiologist, dentist, eye specialist. Doctors can have different abilities and react differently in different situations. The company would recruit the number of doctors according to its requirements. For example, if there is a requirement of only one cardiologist, then the company would appoint only one cardiologist.

2. Write about the issues related to the runtime type identification.

- Runtime type identification or RTTI is a collection of technologies that allows you to identify an item's type at runtime, which is very useful when you deal with runtime polymorphism.

56 C++ INTERVIEW QUESTIONS

The `dynamic_cast`, `typeid` operator, and the `type_info` structure are the three components of RTTI.

Runtime polymorphism has added these items in C++, but that utility comes at a cost.

The `dynamic_cast` operator is more flexible. For using the `dynamic_cast`, the instantiation of an object is must. Furthermore, `dynamic_cast` works only with pointers and references to polymorphic class objects; there must be atleast one virtual function somewhere in the class hierarchy.

5

Classes and Objects

1. What is a class?

- A class is a prototype that defines the abstract characteristics of an object, such as attributes, fields, or properties. It also defines the behaviors of objects, such as methods, operations, or its features. In other words, you can say that classes describe objects.

Classes provide modularity and structure in an object-oriented program. A class is a user-defined data type that may consist of data members and member functions. You can bind data representations and their algorithms into a new data type by defining a class. The syntax to use a class as a data type of an object in C++ is same as the syntax to use the data type of an integer. For example, if Fruit is considered to be a class; mango can be one of the object of this class. The following code snippet shows the declaration of an object, mango, of the Fruit class:

```
Fruit mango;
```

Similar to mango, other objects of the Fruit class can be apple, banana, and orange.

2. How do C++ struct differs from the C++ class?

- C++ defines structures and classes almost identically. To declare a class, you use the **class** keyword in place of the **struct** keyword. The only other differences are related to the default access specifiers. The members of a structure have

public access by default; whereas, the member of a class has private access by default. The concept of encapsulation is supported by the class only and not the structure.

3. Write about the use of the virtual destructor.

- When an object is declared with the new operator and the pointer type is that of a base class with a non-virtual destructor, the base destructor executes instead of the derived destructor. When a base class destructor is virtual, the compiler calls the correct destructor function irrespective of the type of the pointer.

If the base class needs no custom destruction, you must still provide a virtual destructor (with an empty block) to permit the proper destructor calls for dynamically allocated objects. Moreover, by making the destructor of the base class as virtual, you can invoke the destructors of both the base and derived classes in the reverse order.

4. Write a note on encapsulation.

- Encapsulation is an object-oriented design approach that closely binds the implementation of a class to its data representation, logically hides the details of the implementation from users of the data type, and provides a public interface to the data type's behavior. It implies an implementation, which is hidden from the class user, and an interface, which is visible to the class user. It helps you to handle large scale programming tasks. The idea behind encapsulation is to take a complex system that demands a lot of attention and to turn it into an object that handles all its work internally and can easily form a concept.

5. Write a note about this pointer.

- The **this** pointer is a pointer that exists in all non-static member functions. It points to the object for which the function is currently executing.

It is a hidden argument that is passed to every member function as if the member function were declared as shown in this example:

```
void Date :: myFunction(Date* this);
```

The function declaration does not really contain the parameter when you write the code.

When your program calls a member function for an object, the compiler inserts the address of the object into the argument list for the function. For example:

```
dt.myFunction (&dt);
```

Once again, you do not include the object's address as an argument when you write code that calls a member function. This example shows what the compiler generates as the hidden argument for the member function's hidden **this** pointer parameter.

When you call a member function for an object, the compiler assigns the address of the object to the **this** pointer and then calls the function. Therefore, every reference to any member from within a member function implicitly uses the **this** pointer.

6. What are the effects after calling the delete this operator ?

- It is difficult for the compiler to know whether an object is allocated on a stack or a heap. On invoking the **delete this** operator, the destructor is called twice on the object related to the current context. Firstly implicitly and secondly explicitly. It results in undefined behavior and error-prone conditions. The delete this operator can only work on three conditions:

- ☐ An instantiation of a local variable should not take place, after calling the delete this destructor
- ☐ The **this** pointer should not be used after calling the delete this operator
- ☐ The object of a derived class should be allocated and initialized with the **new** operator

7. Write a note about assignment operators.

- Assignment operators are the expressions that return values. The values that they return are the values that are assigned to the variable on the left side of the assignment. The assignment operator has the right to left associativity, which means that the expressions are evaluated starting with the rightmost one. Let's take an example:

```
FirstTotal = SecondTotal = 0;
```

In the preceding example, the `SecondTotal = 0` is evaluated first. This expression returns the result of the assignment, which is zero in this case. When the leftmost expression is evaluated, the rightmost expression is assigned to the leftmost variable within a parenthesis, as shown in the following code:

```
FirstTotal = (SecondTotal = 0);
```

The parentheses are not needed to force the precedence of the rightmost expression over the leftmost expression, because of the associativity of the operator.

8. Write about all the implicit member functions of a class?

- All the implicit member functions of a class are listed as follows:
 - a. *Default constructor* —Defines a constructor with no parameters.
 - b. *Copy constructors* —Initializes the value from an existing object of a class to a new, instantiated object of that same class.

- c. *Assignment operator*—Returns a value.
- d. *Default destructor*—Runs automatically when an object is being destroyed.
- e. *Address operator*—Takes one operand and returns the memory address of a variable or function.

9. Write about the container class and mention its types.

- A container class provides built-in features and procedures that let you handle stored data, find the number of stored objects, insert new objects, and so on. The container class includes general-purpose classes, such as stacks, lists, and queues in which the maintenance of the container is generic but the item in the container is specific.

You can use the basic built-in C++ types as well as objects that have public copy constructors and assignment operators.

There are 11 types of container classes as shown in the following list:

- *List*—Supports a doubly linked list of data items; each item knows which item comes before and after it.
- *Queue*—Supports a queue that lets you add elements at the back end and remove them from the front end. This container is more restrictive than a deque.
- *Deque*—Supports a double-ended queue, and it also lets you access those elements randomly.
- *Priority_queue*—Supports a prioritized queue that stores its items in a sorted order.
- *Stack*—Supports a stack of items, with methods like push and pop.
- *Vector*—Supports a one-dimensional array.
- *Map*—Supports a collection of keys and values associated with the keys. The key and values can be of different types.

- *Multimap* —Supports a map that can support multiple values for each key.
- *Set* —Supports a sorted set of unique members.
- *Multiset* —Supports a set that can support multiple values per key.
- *Bitset* —Refers to a data construct that allows you to set, reset, and check individual bits.

10. What is overriding.

- To override a method, a subclass of the class that originally declared the method must declare a method with the same name, return type (or a subclass of that return type), and same parameter list.

Overriding a method means replacing the method's functionality in a child class. To implement overriding functionality, you need parent and child classes. In the child class, you define the same method signature as the one defined in the parent class.

11. How do you access the static member of a class?

- A program can use the **scope resolution operator (::)** to get access to the static member of a class. You express the call to the base class function by coding the base class name and the double colons ahead of the member name to bypass the overridden member. The following example displays the usage with the calculator class:

```
#include <iostream.h>
#include <conio.h>
class calculator
{
private:
int iNum1, iNum2, iNum3;
public:
calculator();
void input();
```

```
void sum();
void disp();
};
calculator::calculator()
{
    iNum1=iNum2=iNum3=0;
}
void calculator::input ()
{
    cout<<"Enter the first number: ";
    cin>>iNum1;
    cout<<"Enter the second number: ";
    cin>>iNum2;
}
void calculator::sum()
{
    iNum3=iNum1+iNum2;
}
void calculator::disp()
{
    cout<<"The sum of the two numbers is
"<<iNum3<<endl;
}
int main()
{
    calculator A1;
    A1.input();
    A1.sum();
    A1.disp();
    getch();
    return 0;
}
```

In the preceding code snippet, the member functions and the constructors of the class calculator are defined outside the class declaration by using the scope resolution operator(::). The

void is the return type of the function calculator is the class name, (::) is the scope resolution operator, input is the function name, and int iNum1 and iNum2 are the function parameters.

12. Write about a nested class and mention its use.

- A nested class is a class, defined within the scope of another class. Let's take an example:

```
// nested class
classA(any outer class)
{
    classD (nested class)
    {
        // ...
    };
    // ...
};
```

Nested classes are used to organize the codes and control the access of different codes. In the preceding example, if the access specifier is made public for the nested classes, then the name should be `classA (outer class):: classD(nested class)`. If the access specifier is made private, then the nested class can be accessed by the members of the outer class only.

13. Write about the local class and mention its use.

- A local class is defined as the abstract characteristics of an object, such as attributes, fields, or properties. It also defines the behaviors of objects, such as the methods, operations, or its features. In other words, you can say classes describe objects.

A class nested inside the scope of the local helps in managing the codes. Let's take an example:

```
// local class
intD ()
{
    classC (local class)
    {
        // ...
    };
    // ...
};
```

14. Write about the members that a derived class can add.

- Derived classes can use the constructors, destructors, and assignment operator methods of their base classes. It can also override the member functions of the base class. For example, if the base class has an assignment operator method, the compiler uses that method for the derived class as well, which is fine unless the derived class also adds its own data members (in which case, you should create a **new operator=** method for the derived class).

15. Describe the process of creation and destruction of a derived class object.

- When you declare an object of a derived class, stacks or heaps allocate the space for the object. This space contains the inherited data members from the base class and also the members defined in its derived class. The initialization of the inherited data members from the base class is done with the help of the constructor of a base class. Therefore, compiler executes the constructor function of the base class followed by the constructor function of the derived class.

When an object goes out of scope, the destructors execute in the reverse order of the constructors. Firstly, the destructor of the derived class is invoked to destroy the derived class object, and then the destructor of a base class is invoked. This process helps to allocate the required space.

16. Explain the process of creating a subscript operator for a matrix class.

- The `operator()` method is used instead of the `operator[]` method when you have multiple subscripts. The `operator()` method can take any number of parameter methods. On the other hand, the `operator[]` method can take only one parameter. Let's take an example:

```

#include <iostream.h>
#include <conio.h>

class MyMatrix {
public:
    MyMatrix(unsigned rows, unsigned cols);
    double& operator() (unsigned row, unsigned col);
    double operator() (unsigned row, unsigned col)
    const;
    ~MyMatrix();
    MyMatrix(const MyMatrix& m);
    MyMatrix& operator= (const MyMatrix& t);
private:
    unsigned rows_, cols_;
    double* data_;
};

inline MyMatrix::MyMatrix(unsigned rows, unsigned
cols): rows_ (rows), cols_ (cols)

{
    if (rows == 0 || cols == 0)
        throw("error: MyMatrix constructor has 0 size");
    data_ = new double[rows * cols];
}

inline MyMatrix::~~MyMatrix()
{
    delete[] data_;
}

inline double& MyMatrix::operator() (unsigned row,
unsigned col)
{
    if (row >= rows_ || col >= cols_)
        throw("error:Matrix subscript out of bounds");
    return data_[cols_*row + col];
}

inline double MyMatrix::operator() (unsigned row,
unsigned col) const
{
    if (row >= rows_ || col >= cols_)
        throw("error:constMyMatrix subscript out of
bounds");
    return data_[cols_*row + col];
}

int main()

```

```
{  
    MyMatrixt(12, 12);  
    t(7,10) = 108.16;  
    cout<< t(7,10);  
    getch();  
}
```

17. Give reason why the interface of the MyMatrix class should not look similar to array-of-array.

- TheMyMatrix class with operator[] always returns an array object's reference and an element of the MyMatrix, such as a double's reference is returned by the object of an array with an operator[]. Therefore, accessing of elements of the matrix is done by using the syntaxt[j][k] instead of t(j,k).

The operator() method is more flexible than array-of-array[[]]. This method is easy in its implementation. On the contrary, the operator[[]] method is specific in its use, which means it is used only in the case of the dense matrix.

Let's take an example. In case of bigger rows and columns (logical layout) which are almost of the size of the cache (block of Random Access Memory, used to store data), the retrieval of values from each rows and columns takes a long time and degrades the performance. On the other hand, using the physical layout (column , row) helps in retrieving the source codes files of columns by the rows, swap the ordering of the row/column using the operator() and improves the performance. Therefore, it is better to use physical layout(column, row).

18. Write about the use of the static data members.

- The C++ compiler allocates only one memory location for the static data member, no matter how many objects you instantiate. When you declare a method static, it becomes a class (not object) method, which means you can invoke it simply with the class name. This is very useful if you want all the objects of a certain class to share some specific data and

can provide a way of communicating between those objects. There are two advantages of static data members— you can declare them to be of the same class of which they are members (which you can't with other data members) or you can use them as a default argument to a method of the class of which they are a part.

Let's take an example. There is a class named `bird_classes`. The `number_seen` data member of this class is made static. It means that all objects of this class will share the same value in `number_seen`. In this way, you will be able to keep a track of the total numbers of birds seen by all objects. Let's understand this concept with the help of the following code snippet:

```
#include <iostream.h>
#include <string>
using namespace std;
class bird_class
{
public:
    static int number_seen;
    string name;
    void increment_count(){number_seen++;}
    bird_class(string text);
};
```

You need to initialize static variables outside the class definition; otherwise, your compiler will throw an error that there has been no memory allocated for that variable. In the following coding, you initialize the `number_seen` outside the class definition by using a scope resolution operator `::`:

```
#include <iostream.h>
#include <string>
using namespace std;
class bird_class
{
public:
    static int number_seen;
    string name;
    void increment_count(){number_seen++;}
```

```
bird_class(string text);  
};  
int bird_class::number_seen = 0;
```

Now, you can create two objects of `bird_class`, increment the number of each object of the `bird_class`, and then the output will display "number of birds seen: 2".

```
#include <iostream.h>  
#include <string>  
#include <conio.h>  
using namespace std;  
class bird_class  
{  
public:  
    static int number_seen;  
    static name;  
    void increment_count(){number_seen++;}  
    bird_class(string text);  
};  
  
int bird_class::number_seen = 0;  
  
bird_class::bird_class(string text)  
{  
  
    number_seen = 0;  
}  
  
int main()  
{  
    bird_class orioles("oriole");  
    bird_class robins("robin");  
  
    orioles.increment_count();  
    robins.increment_count();  
  
    cout<< "Number of birds seen: " <<bird_class ::  
    number_seen<<endl;  
    getch();  
    return 0;  
}
```

19. What is an object?

- An object is a declared instance of a data type, which includes standard C++ data types as well as objects of classes. In other words, they are the variables that you declare in the class. In OOP, objects represent real world entities, such as students, doctors, and bank accounts. These objects can interact with each other by sending or receiving messages. An object has the following three main characteristics:
 - ☐ It has a state
 - ☐ It may or may not display a behavior
 - ☐ It has a unique identity
-

20. Define a good interface.

- A good interface is the one that hides unnecessary details and provides a simplified way for making applications with all the required information and different software related to different applications. A good interface should provide users an opportunity to explore and understand the different concepts related to the tasks they want to do. In this way, it ensures good communication between the user and the computer.
-

21. Write about the role of C++ in the tradeoff of safety vs. usability.

- Earlier in C, an encapsulation is achieved by making the methods static in a class. On the contrary, in C++, it would not work.

Programmers use structs in C for making multiple instances. The creation of multiple instances is not supported directly by the data with static keyword in a class. This situation made the concept of tradeoff between safety (information hiding) and usability (multiple instances) worse.

On the contrary, C++ supported both multiple instances and encapsulation with the help of the concept of a class. The public part of a class includes the interface of the class which includes public member functions and friends functions of a class. The private and/or protected parts of a class includes an implementation of the class which includes the data. This results in a struct which is encapsulated. This depicts the strong tradeoff between safety (information hiding) and usability(multiple instances).

22. How can you prevent accessing of the private parts of my class by other programmers (violating encapsulation)?

- Encapsulation only hides the details of the implementation from users of the data type, and provides a public user interface. Encapsulation only maintains the integrity of the internal coding inside the methods of a class, but not able to prevent accessing the private or protected parts of a class by the user.

23. Can non-public members of another instance of the class be retrieved by the method of the same class?

- Yes, the non-public members of another instance of the class can be retrieved by the method of the same class. Retrieval depends on the type of the class whether it is of a reference/pointer/object class. It seems to break the rule of encapsulation. However, there is a need of get () method with a public specifier for the non-public members to retrieve the method of the same class because an explicit argument (means the pointer by which "this" is not called) can be taken by at least one method of the same class.

24. Can encapsulation be called as a security device?

- Encapsulation hides important and classified data from accidental manipulation by members of the same program. Note that the encapsulated data of a class can be accessible

outside the class only when the data is used in the function of the class. However, a limited number of operations can be performed on encapsulated data by executing the functions or methods of the class. An encapsulation prevents the internal error to occur and not the explicit attacks or intentional attacks and therefore cannot be called as a security device.

25. How the keyword struct is different from the keyword class in C++?

- In C++, a class is similar to a struct with the exception that, by default, all the members of a class are private; while the members of a struct are public. Encapsulation is not supported by structures but supported by classes.

6

Memory Allocation/Deallocation

1. How new/delete differs from malloc()/free?

- The memory uses the malloc() operator, which in turn uses the free operator, to remove the unnecessary programs. On the other hand, the program builds the array by using the new operator, fills it with random numbers, displays each of the elements in the array, and deletes the array by using the delete operator. The new and delete operators should be used in C++ because they are type safe. The malloc operator is used when there is a need of forcing a type on an object because a void pointer is returned by it. Moreover, that object cannot be assigned to other types.

2. How the delete operator differs from the delete[] operator?

- When the new[] operator is used to allocate memory dynamically, the delete[] operator is used to free the memory. The new[] operator is used to allocate memory to an array of values, which starts with the index 0.

3. How a new operator differs from the operator new?

- The new operator creates a class's new instance. On the other hand, overloading of a new operator is done globally with the help of the operator new. The new operator allocates memory for the item and assigns the address of that memory to the pointer by using the name of an item with a pointer of a data type, structure, or array. For example, consider the following code snippet:

```
double * pi = new double;
```

In the preceding code snippet, the new operator returns a pointer to the **double** variable, because it allocates the space for a **double** value.

4. Explain the term memory alignment.

- The primary meaning of the term alignment is to maintain the appropriate positioning of different components in the memory with respect to each other. In C++, there is a requirement of setting of various objects and variables in a particular way in the system's memory. Therefore, many data variables are aligned automatically by the compiler according to their processor and type.

5. Explain the concept of dynamic allocation of memory.

- In C++, memory is divided into three areas, automatic storage (where function's variables are allocated and stored), static storage (where static data is stored and left for the duration of the program's runtime) and the free storage (the memory set for the use of your code at runtime; the new and delete operators are used to allocate and handle space in the free storage).

Allocating the memory dynamically means that you can allocate space for as many elements as you need at runtime. The memory can be allocated dynamically with the new operator and free with the delete operator. Let's understand it with the help of an example-

```
#include<iostream.h>
#include <conio.h>
int main()
{
    double * pi = new double;
    *pi = 3.14159;
    cout<< "The value of pi = "<<*pi<<endl;
    delete pi;
    getch();
    return 0;
}
```

In the preceding code snippet, allocation of memory is done for a double variable, `pi`, by using the `new` operator. This operator returns a pointer to a double variable.

6. In the expression, `char *c = (char *)malloc(20);` followed by `free(r);` explain the role of `malloc()` operator in allocating the number of bytes?

- Implementation is the main factor associated with the allocation of number of bytes with the help of the `malloc()` operator. The `malloc()` operator includes the number of bytes allocated + 4 bytes. In the given expression, `malloc()` allocates 24 bytes. 4 bytes is used for storing the number of bytes allocated. It means 20 bytes can be accessed from `char`.

7. How the allocation of two-dimensional arrays is done by using `new`?

- The allocation of a two-dimensional array using `new` depends on the size of an array. An array can be allocated statically, on determining all the dimensions at the time of compilation. Let's understand this concept with the help of the following code snippet:

```
#include <iostream.h>
#include <cstring.h>
#include <conio.h>

int main(int argc, char *argv[])
{
    unsigned i;

    // Declaration of the two-dimensional array
    // as a pointer to pointer
    //
    char** array_2D;
    unsigned A = 20;
    unsigned B = 30;

    // Allocate "main" array
    //
    array_2D = new char*[A];

    // Allocate each member of the "main" array
```

```

//
for (i = 0; i < A; ++i)
array_2D[i] = new char[B];

// Fill the 6th element with a string and
// print it out
//
strcpy(array_2D[7], "welcome");
cout<< array_2D[7] <<endl;

// Deletion is performed in reversed order.
// Pay special attention to the delete[]
// operator which must be used to delete
// arrays (instead of the "simple" delete)
//
for (i = 0; i < A; ++i)
delete[] array_2D[i];

delete[] array_2D;
getch();
return 0;
}

```

In the preceding code snippet, the two-dimensional array is allocated memory by using new.

8. Is it possible to use the free operator with the new allocated pointers and the delete operator() with the malloc allocated pointers?

- It is not possible to use the free operator with the new allocated pointers and delete with the malloc allocated pointers because it is illegal. On the other hand, to use malloc() and delete together for the same program or the use of new and free() for the same program is legal.

The incorrect use of these operators may lead to the error-prone conditions to occur if a new machine, a new compiler, or the same compiler's new version is used.

9. Can a new be used in place of old malloc()? If yes, why?

- The new operator should be used in place of old malloc because the new operator ensures the calling of an appropriate destructor at the time of execution and also it is more type-safe than malloc().

10. Is it possible to use a new for the reallocation of pointers ?

- The reallocation of pointers cannot be done by using new. It can be done by using the realloc() operator.

11. Is there any need for checking of NULL after executing p = new Animal()?

- There is no need for checking of NULL after p = new Animal() because the repetitive use of an explicit NULL after the allocation of new makes the code very monotonous.

Let's understand this concept with the help of the following code:

```
Animal* p = new Animal();  
if (p == NULL)  
    throw std::bad_alloc();
```

Moreover, in the absence of the support of the compiler, the code becomes more monotonous.

For example,

```
Animal* p = new Animal();  
if (p == NULL) {  
    std::cerr<< "Couldn't allocate memory for an  
    Animal" <<std::endl;  
    abort();  
}
```

When the allocation of sizeof(Animal) during p = new Animal() is not done, an exception indicating the bad allocation will be thrown. There should be a simple coding which is:

```
Animal* p = new Animal();//the checking of NULL is  
not needed.
```

In .cpp files, some standard headers must be included in case of failing of throwing an exception by new.

12. Explain the process of returning of a NULL by the old compiler after the checking of new.

- The old compiler will return NULL after checking for the new operator, but not implicitly.

The NULL test is explicitly done to install the new handler function. Many functions can be performed with the help of new handler function, such as throwing an exception, printing

a message, and deleting some objects. Let's take an example, there is a new handler that throws an exception. The installation of this handler is done by using `std::set_new_handler`. Let's understand this concept with the help of the following code snippet:

```
#include // To get std::set_new_handler
#include // To get abort()
#include // To get std::cerr

class alloc_error : public std::exception {
public:
    alloc_error() : exception() { }
};

void ourNewHandler()
{
    // This is your own handler. It can perform many
    // functions.
    throw alloc_error();
}

int main()
{
    std::set_new_handler(ourNewHandler); // Install
    your "new handler"
}
```

After the execution of `std::set_new_handler()` line, `ourNewHandler()` will be called by operator `new`.

13. Can a NULL be checked before deleting the p variable from the expression, Delete p; and not if (p != NULL) ?

- NULL needs not to be checked before deleting the p variable because if the p variable is equal to NULL, it does not affect the delete p statement.

14. Enlist the processes involved in deleting a variable.

- There are two processes involved in deleting a variable, which are as follows:
 - ☐ Calling a destructor
 - ☐ Releasing the memory

15. On throwing an exception by the Animal constructor in `p = new Animal()`, can memory leak occur?

- Memory of an animal class cannot leak by throwing an exception in `p = new Animal()`. In case of occurrence of an exception during the Animal constructor of `p = new Animal()`, there is a surety of the automatic releasing of the allocated memory back to the heap.

16. What is the use of `[]` in the delete statement of the following expression:

```
Use p = new T[n] and delete[] p:  
Animal* p = new Animal[80];  
...  
delete[] p;
```

- In the preceding expression, `[]` in the delete statement is used to allocate an array using the new(also called `[n]` expression).

17. What would happen on forgetting `[]`, while deallocating an array through new?

- If you forget to use `[]` while deallocating an array through new, it throws a run time or compile time exception and results in the corruption of the heap. Therefore, it is the responsibility of the programmer to establish the connection between `T[n]` and `delete[]p` correctly.

18. Can `[]` be dropped on deletion of some built-in type array (char, int etc)?

- `[]` cannot be dropped on deletion of some built-in type array (char, int) because the use of delete instead of delete[] can cause run time error in a program. Particularly, the operator called for delete is delete (void*).

19. Write about the retrieval of n number of objects during the process of delete[]p?

- The n number of objects can be retrieved with the help of the pointer p (it stores the n number of objects) at the run time.

This can be achieved by the application of two techniques, which are:

1. An array which can be associated with p in the form of key and a value such as n.
2. Over-allocation of the array can be done.

The preceding techniques are used by the compilers of commercial grade.

20. Is it possible for a member function to use delete this?

- It is possible for a member function to use delete this but on certain conditions which are as follows:
 1. Ensure that the allocation of this object is done through new[] operator.
 2. Ensure that the invocation of a member function done on this (current) object will be the last member function.
 3. Ensure that calling of other member functions and data members should not be done after the line of code which includes delete this.
 4. Examination or comparison of this pointer with other pointers and NULL, printing or casting it, must be avoided after using the delete this.

The preceding points are applied in the cases where the this pointer belongs to the base class in the absence of the virtual destructor.

21. Discuss the effects occur, after an exception thrown by a member function is unspecified by an exception specification.

- When a function throws an exception, not given in the specification, the exception is passed to a system function named unexpected. The unexpected function calls the latest function named as an argument in a call to the set_unexpected function, which returns its current settings. A

function with no exception specification, by default, calls the terminate function, which finally calls the abort ()function to terminate the program.

22. Explain the throw statement with its use.

- The throw statement in C++ specifies an error and displays the An error has occurred message. The throw statement is used to unwind the stack. Moreover, with the help of destructors, it cleans up the objects that are declared inside the try block. Next, the throw statement called the catch handlers which are matched with the object passed in the parameter list. Let's understand this concept with the help of the following code snippet:

```
#include <iostream.h>
#include <conio.h>
void abc();
class xyz {}; // an exception to be thrown
class waste {}; // an exception to be thrown

int main()
{
//...the try block
try
{
cout<< "calling abc" <<endl;
abc();
cout<< "return from abc<<endl";
}
//...catch exception handler
catch(xyz)
{
//error-handling code
cout<< "catching xyz" <<endl;
}
catch(waste)
{
//error-handling code
cout<< "catching waste" <<endl;
}
cout<< "done" <<endl;
return 0;
}
```

```
//...program function
void abc()
{
    int error = 1;
    //C++ statements to do stuff
    if (error)
    {
        cout<< "throwing waste" <<endl;
        getch();
        throw waste();
    }
}
```

In the preceding code snippet, the functions which are called directly or indirectly from the try block can throw exceptions. The catch exception handler catches exceptions. The waste class is set up specifically to identify the exception.

23. How can i be thrown polymorphically?

- The following code can be used to throw i polymorphically:

```
class ExceptionParent { };
class ExceptionChild : public ExceptionParent { };
void f(ExceptionParent& e)
{
    // ...
    throw e;
}
void g()
{
    ExceptionChild e;
    try {
        f(e);
    }
    catch (ExceptionChild& e) {
        //...code to handle ExceptionChild...
    }
    catch (...) {
        //...code to handle other exceptions...
    }
}
```

In the preceding code, you can enter the catch (...) clause in the absence of throwing i polymorphically. The throw e in the function f () throws the same type of an object as an expression of static type.

24. What is the role of copy constructor in copying of thrown objects?

- A copy constructor with a public access specifier must be applied to the objects which are thrown. With the help of a compiler, the code is generated through which copying of thrown objects can be done. The thrown objects must have a copy constructor with the public access specifier.

25. What is the role of Microsoft Foundation Class (MFC) in relation to encourage the use of catch-by-pointer?

- The compiler of Microsoft is slow in supporting the C++ exceptions. On the contrary, the standardization of the C++ exception handling is done in MFC (library). It implements the applications on a particular operating system platform, such as Microsoft windows. MFC uses the backward-compatibility in relation to exception handling. The use of catch-by-pointer is encouraged by some of these libraries.

Let's consider the following code snippet:

```
Exception6 m;  
voidg()  
{  
    Exception6 n;  
    try {  
        switch (rand() % 3) {  
            case 0: throw new Exception6;  
            case 1: throw &m;  
            case 2: throw &n;  
        }  
    }  
    catch (Exception6* p) {  
        //... should the p be deleted?  
    }  
}
```

There are three basic problems related to the use of catch-by-pointer which are listed in the following order:

1. When the object is within the private part of a class or becomes static within some compilation unit, it becomes difficult to delete the variable within the catch block.

2. The new operator in throw and the delete operator in catch always use the heap, which creates problems regarding the running of low memory.
3. The new operator in throw and the delete operator in catch are unable to allocate the local objects because the destruction of them may occur earlier. To avoid the destruction, functions, such as thread-safety and locks, should be included.

26. How can an improvement in the quality of software be done by try/catch/throw?

- Error-handling is necessity while developing applications to account for unexpected situations, such as insufficient memory, resource allocation errors, inability to find/open files, division by zero, an arithmetic or array overflow, and the exhaustion of free heap space, which occur at runtime.

Programmers have various styles of dealing with such exceptions, such as try/catch /throw methods, which cause diversity in coding practice. This diversity increases with the use of user-defined classes, as each class brings with it potential class-specific errors. These methods provide a standard facility to deal with runtime exceptions. Moreover, try/catch/throw result in the code with fewer errors. It is low in cost in relation to development. With error-handling, your program can detect unexpected events and recover from them.

27. Explain the process of throwing variables.

- You can throw exceptions by using variables. This is the better way to support exceptions, because you can add your own methods and data members to these variables.

The following code snippet depicts a temporary variable named exception:

```
#include <iostream.h>
#include <string.h>
#include <conio.h>

class Exception
{
    stringerror_message;
public:
    stringget_error(){return error_message;}
    Exception(string s);
};
Exception::Exception(string s){error_message = s;}
int main()
{
    float op1, op2;
    cout<< "Enter the two floating point numbers to
        divide:";
    cin>> op1 >> op2;
    try
    {
        if(op2 != 0)
            cout<< op1 <<"/" << op2 << " = " << op1/op2 <<endl;
        else
            throw Exception("Error: division by zero.");
    }
    catch (Exception e)
    {
        cout<<e.get_error() <<endl;
    }
    getch();
    return 0;
}
```

In the preceding code snippet, a class named `Exception` can hold an error message in an internal data member and has a constructor and one method `get_error`, which returns the error message. Now, when an error occurs in the `try` block, the object of the `Exception` class can be created by passing an error message to this class's constructor. In the `catch` block, the `get_error` method is used to display that error message.

28. Explain the process of catching a variable.

- The options available in C++ for catching a variable are as follows:

- ☐ catch by value
- ☐ catch by reference
- ☐ catch by pointer

In fact, catch by value should be avoided because different behavior can be exhibited by the two copies of exception objects with respect to the throwing object.

In the case of catch by reference, firstly, the copying of objects of exception is prevented. Secondly, if any modification made in exceptions which are within the catch-clause is retained on re-throwing. Therefore, catch by reference is preferred as compared to catch by value or catch by pointer.

In the case of catch by pointer, an exception with pointer can be caught. For example- D and D* are of different types. Only value can be caught on throwing the pointer to D. Catching by value is done on D* and not D. Therefore, the use of catch by pointer should also be avoided.

7

Pointers

1. What is a dangling pointer?

- A: When the location of the deallocated memory is pointed by the pointer even after the deletion or allocation of objects is done, without the modification in the value of the pointer, then this type of pointer is called a dangling pointer.

2. Explain the concept of memory leak.

- When a variable does not exist longer in the memory, and deletion or reuse of that variable cannot be done, then its destruction occurs automatically. This concept is called memory leak.

For example, consider the following code snippet:

```
{  
    Parent*p = new parent ();  
}
```

In the preceding code snippet, the p variable does not exist in the memory, but the variable is not deleted. Therefore, it becomes out of scope and its destruction occurs, which results in memory leak.

3. What is an auto pointer?

- When you allocate some memory to a variable and that variable becomes out of scope, then the deallocation of the memory can be done with the help of an auto pointer. This reduces the need of using the delete operator to deallocate memory. To use auto pointer, you can use the syntax

`auto_ptr<type> name` (new type) and include the memory header file in which `auto_ptr` is defined. Let's understand the use of auto pointer with the help of the following code snippet:

```
#include <iostream.h>
#include <memory.h>
#include <string.h>
using namespace std;

int main(int argc, char **argv)
{
    int *j = new int;
    auto_ptr<int> m(j);
    auto_ptr<int> n;
    y = x;
    cout<<m.get() <<endl;
    cout<<n.get() <<endl;
}
```

In the preceding code snippet, the pointer `j` is deleted automatically by `auto_ptr`.

4. List the issue that the `auto_ptr` object handles.

- The `auto_ptr` object is used to deallocate memory, which is allocated to a variable, when the variable goes out of scope.

5. What are smart pointers?

- Smart pointers are almost similar to pointers with additional features, such as automatic destruction of a variable when it becomes out of scope and the throwing of exceptions that ensures the proper destruction of the dynamically allocated objects. They are useful in keeping tracks of dynamically allocated objects. Due to these additional capabilities, they reduce the possibilities of occurrence of exceptions and errors in a program and ensure that the written code is safe and efficient.

6. Write about the complexities related to the following code:

```
int*b = NULL;
int&p = *b;
```

- The preceding code results in an undefined behavior as a reference pointer, `&`, cannot contain `NULL`.

7. How a pointer differs from a reference?

- A pointer differs from a reference in the following ways:
 1. In the case of reference, an object must always be referred while initializing. On the contrary, such restrictions are not meant for pointers.
 2. The different types of objects can be pointed by the pointers by reassigning the pointers with different objects. On the contrary, in a reference, the same object which was initialized earlier can only be referred by a reference.
 3. You can use a null address in a pointer parameter to indicate that a variable does not exist; whereas, there is no existence of a null reference in C++.
 4. A reference usually appears outside an object; whereas, a pointer generally appears inside an object.

8. How `const int *ourPointer` differs from `int const *ourPointer`?

- As a rule, pointer declarations should be read from right to left. In the pointer declaration, `const int *ourPointer`, `ourPointer` is a pointer to a `const int` object, which cannot be changed by using a pointer. Whereas in case of the `int const *ourPointer` pointer declaration, `ourPointer` is a `const` pointer to an `int` object, in which an `int` object can be changed by using a pointer but the pointer itself cannot be changed because it is constant.

9. Mention the conditions in which we should use references and pointers separately.

- While creating a pointer, you need to declare a pointer variable and then store the address of another variable in it. A pointer variable can directly access the value stored in the other variable by using the * operator, as shown in the following code snippet:

```
cout<< *ptr;
```

In the preceding code snippet, the * operator informs the compiler to access and display the value stored at the memory location referenced by the ptr pointer variable.

The * operator is also known as the dereference operator. The dereference operator (*) accesses the data stored in the variable pointed by the pointer variable.

The reference variable is generally used to simplify the dereferencing of the complex objects, which are found inside arrays, structures, arrays of structures, and array inside structures.

8

Strings

1. What happens on executing the following code?

```
string&alphabets()  
{  
    return "welcome";  
}  
cout<<alphabets() <<endl;
```

- On executing the preceding code, the following error occurs:
The string, Welcome, is unnamed because the constant variable is not declared and it is assigned to the non-constant reference, &alphabets, which is not allowed.

2. Explain the ways in which type conversion can be done.

- Type conversion refers to the process of assigning a value of one data type to a variable of another data type. There are two types of conversions- implicit and explicit. Implicit conversions can be handled by the compiler and the explicit conversions can be handled by a user. Following are the ways by which type conversion can be performed in C++:
 1. *const_cast <type> (expression)*—Allows you to convert a variable's data type by adding a const or volatile modifier to it
 2. *dynamic_cast <type> (expression)* —Supports runtime identification of class objects
 3. *reinterpret_cast <type> (expression)* —Supports low-level reinterpretation of the bit pattern of an expression

4. *static_cast* *<type>* (*expression*) —Makes explicit conversions of types into non-equivalent types, such as conversion of pointers from base class to derived class or vice-versa

9

Overloading Operators

1. List the operators that cannot be overloaded.

- The operators that cannot be overloaded are as follows:
 - ☐ . - Member selection operator
 - ☐ .* - Pointer-to-member selection operator
 - ☐ sizeof() operator
 - ☐ :: - Scope resolution operator
 - ☐ ?: - Conditional operator

2. Explain how overloading takes place in C++.

- C++ supports two types of overloading namely, function overloading and operator overloading. Function overloading helps in defining more than one function with the same name, but with different signatures. An error is raised if two overloaded functions are provided with the same function signature. Operator overloading helps in giving special meanings to operators, when they are used with user-defined classes.

3. What is the difference between prefix and postfix versions of operator++()?

- The prefix and postfix versions of operator ++() can be differentiated on the basis of arguments defined. The postfix operator ++() consists of a dummy parameter of int datatype; whereas, a dummy parameter is not found in the prefix operator ++().

4. Can a function overloading depend only on passing by value and passing by reference ?

- No, the reason is that whether a function is called by passing a parameter as a value or by reference, it appears similar to the caller of the function.

5. What's the use of operator overloading?

- Operator overloading helps in providing a built-in interface to the users of a class. It helps a class and function templates to work efficiently with different classes and data of different types. It also allows C++ programmers to create user-defined functions on user-defined classes. Let's understand the concept of operator overloading by creating a program. This program changes positive numbers into negative and negative numbers into positive by using the operator overloading concept. The following code snippet shows the code of the program:

```
#include<iostream.h>
#include<conio.h>

class Change
{
    int p,q,r;
public:
    void getoriginaldata(int j,int k,int l);
    void getthemodifieddata();
    void operator -();
};

void Change::getoriginaldata(int j,int k,int l)
{
    p=j;q=k;r=l;
}

void Change::getthemodifieddata()
{
    cout<<"The first element: " << p<<"The
second element: " << q<< "The third
element: " <<r;
}

void Change::operator -()
{
    p=-p;
    q=-q;
    r=-r;
}
```

```
int main()
{
    clrscr();
    Change C;
    C.getoriginaldata(60,70,-80);
    C.getthemodifieddata();
    cout<<endl;
    -C;
    C.getthemodifieddata();
    getch();
}
```

6. Describe the advantages of operator overloading.

- Operator overloading is used to provide some extra features, behaviors, and abilities to the users of a particular class. This feature in C++ helps in controlling the functions performed by an operator and reduces the chance of occurrence of errors in a program.

7. Provide some examples of operator overloading.

- The implementation of operator overloading is done in the following ways:
 1. Concatenating two `std::string` objects by using the `+` operator
 2. Incrementing a `Date` object by using the `++` operator
 3. Multiplying two different number objects by using the `*` operator
 4. Accessing array elements from an object of the `Array` class by using the subscript operator

8. Operator overloading is supposed to make the code clearer and more understandable but it makes the code more complex to understand. Why?

- An important objective of operator overloading is to make the code clearer and logical for the users of a class. Operator overloading facilitates passing of different parameters to the same function and getting different outputs with the different

parameters. Let's understand this with the help of the following code snippet:

```
class B
{
public:
    Int& operator[] (unsigned d); // Many programmers
    avoid using this syntax
};
inline;
Int& B::operator[] (unsigned d) //This syntax is
also avoided by some programmers
{

}

int main()
{
    B abc;
    abc[4] = 5; // This code is more clear and can
    easily understood by the user
}

}
```

In the preceding code snippet, the B class is declared with abc as an object. The abc object calls an array of 4 numbers.

9. Identify the operators that cannot be overloaded. Provide an example using any one of the operator.

- Most of the operators supported by C++ can be overloaded. There are some operators of C language that cannot be overloaded in C++. These operators are . , ?:, and sizeof operators. C++ also contains some operators, such as :: and .*, which cannot be overloaded.

Let's understand overloading of subscript operator with the help of the following code snippet:

```
#include<iostream.h>
#include<conio.h>

class product
{

int number[3];
public:
    product(int count0,int count1,int count2);
    int operator[](int index)
    {
        return number[index];
    }
}
```

```

    }
    };
    product::product(int count0, int count1, int
count2)
    {
        number[0] = count0;
        number[1] = count1;
        number[2] = count2;

    }

    int main()
    {
        clrscr();
        product oranges(200,300,400);
        cout<<"Number of"<<": " <<oranges[0]<<"    "
<<oranges[1]<<"    " <<oranges[2]<<"    " <<
oranges[3]<< endl;
        getch();
        return 0;

    }

```

10. Can the operator == be overloaded for comparing two arrays consisting of characters by using string comparison?

- The operator == cannot be overloaded to compare two arrays consisting of characters using string comparisons. It should be noted that out of the two operands of an overloaded operator, at least one operand should be of user-defined type. This user-defined type usually refers to a class. Two characters can be compared easily using classes, such as **std::string**, rather than using an array containing characters.

11. Can the creation of operator be allowed to perform the to-the-power-of operations?**

- No, you cannot create operator** for to-the-power-of operations. The number of parameters taken by an operator, names of the operators, priority level of the operators, and the associativity of the operators depend on language in which they are being used. The operator** is not present in C++, so it cannot be created.

12. What is the main purpose of overloading operators?

- The main purpose of operator overloading is to minimize the chances of occurrence of errors in a class that is using the overloaded operators. It also helps in redefining the functionalities of the operators to improve their performance. Operator overloading also makes the program clearer, readable, and more understandable by using common operators, such as +, =, and [].

13. Specify some guidelines that should be followed while overloading operators?

- Following are some of the guidelines that should be followed while overloading operators:
 1. Subscript bracket operators should be used for overloading when there is a need of fetching data from the container class.
 2. Arithmetic operators should be used for providing the numerical calculation to the operators.
 3. Comma operator overloading should be used less often as the ordering properties of this operator vary before and after overloading. This variation in the ordering properties confuses the users of this operator.
 4. The priority order in which operators are executed cannot be modified by overloading the operators.
 5. The overloaded operators must follow the syntax of the language in which they are used.

10

Algorithms

1. Write a code-snippet to check if the given linked list is in the ascending order?

- The following code-snippet checks whether or not the given linked list is in the ascending order:

```
//int dataatnode;
node *temporary1; //creating a temporary node
node *temporary2;
temporary1 = head; // Assigning temporary1 as the
first node in the linked list
while (temporary1->next) //next refers to the next
node in the linked list
{
    if(temporary1->dataatnode >temporary2->dataatnode)
    // Checking //whether data at first node is greater
    than the data at next node
    return false;
    temporary1= temporary1->dataatnode;
}
return true;
}
```

2. Write a program to reverse the elements in a link list.

- The following program reverses the elements in the linked list:

```
#include <iostream.h>
#include<string>
#include<conio.h>
using namespace std;

struct LLNODE
{
    int LLData;
```

```

        LLNODE * nextnode;
    };

    void OriginalData(LLNODE * firstnode)
    {
        LLNODE * current=firstnode;
        while (current)
        {
            cout << current->LLData << ' ';
            current=current->nextnode;
        }
        cout << endl;
    }

    void Reverseorder(LLNODE * current, LLNODE *
previous=0)
    {
        if (current->nextnode) Reverseorder(current
>nextnode,current);
        current->nextnode=previous;
    }

    int main()
    {

        LLNODE * n1=new LLNODE; n1->LLData=1;
        LLNODE * n2=n1->nextnode=new LLNODE; n2
>LLData=2;
        LLNODE * n3=n2->nextnode=new LLNODE; n3
>LLData=3;
        LLNODE * n4=n3->nextnode=new LLNODE; n4
>LLData=4;
        LLNODE * n5=n4->nextnode=new LLNODE; n5
>LLData=5;

        n5->nextnode=0;

        OriginalData(n1);
        Reverseorder(n1);
        OriginalData(n5);

        getch();
    }

```

3. Write a program to implement Binary Heap in C++.

- The following program implements binary heap:

```
#include<iostream.h>
#include<stdlib.h>
class HeapSort
{
private:
    int NumberofElements,Size_of_Heap;
    double *MinHeapCondition;
public:
    void insert_Elements(double);
    double DELETE_HEAP();
    void Enter_Elements();
    void Get_Output();
};

void HeapSort::insert_Elements(double
NumberofElements)
{
    if(Size_of_Heap>=65)
    {
        cout<<"Array is overflow.Please enter a number
less than 65.";
        exit(0);
    }

    MinHeapCondition[++Size_of_Heap]=NumberofElements;
    //Setting the order of the heap again
    int S=Size_of_Heap;
    while(S>1)
    {

        if(MinHeapCondition[S]<MinHeapCondition[S/2])
        {
            double t=MinHeapCondition[S];

            MinHeapCondition[S]=MinHeapCondition[S/2];
            MinHeapCondition[S/2]=t;

            S/=2;
        }
        else
            break;
    }
}
```

```

double HeapSort::DELETE_HEAP()
{
    if(Size_of_Heap<1)
        return -1;
    double val;
    val=MinHeapCondition[1];
    MinHeapCondition[1]=MinHeapCondition[Size_of_Heap];
    Size_of_Heap--;

    //Setting the order of the heap again by traversing
    down
    int S=1;
    int Child_S;
    while(2*S<=Size_of_Heap)    //S has a child
    element
    {
        if(2*S==Size_of_Heap)    //if S has only left child
        {
            Child_S=2*S;
        }
        else                    //If S has two children
        {
            if(MinHeapCondition[2*S]<MinHeapCondition[2*S+1])
                Child_S=2*S;
            else
                Child_S=2*S+1;
        }

        if(MinHeapCondition[S]<MinHeapCondition[Child_S])
            break;
        else
        {
            double tr;
            tr=MinHeapCondition[S];

            MinHeapCondition[S]=MinHeapCondition[Child_S];
            MinHeapCondition[Child_S]=tr;

            S=Child_S;
        }
    }
    return val;
}

```

```
void HeapSort::Enter_Elements()
{
    cout<<"-----\n"
    <<"This program can be used to sort elements in \n"
    <<"the ascending order using heap technique\n"
    <<"-----\n";

    cout<<"Enter the total number of elements you want
    to sort: ";
    cin>>NumberOfElements;
    MinHeapCondition=new double[NumberOfElements+1];

    //Constructing the required heap

    Size_of_Heap=0;
    cout<<"Enter the elements\n";
    double number;
    for(int i=1;i<=NumberOfElements;i++)
    {
        cin>>number;
        insert_Elements(number);
    }
}

void HeapSort::Get_Output()
{
    cout<<"Elements in the sorted order are ::\n";
    for(int i=1;i<=NumberOfElements;i++)
        cout<<DELETE_HEAP()<<'\\t';
    cout<<endl;
}

int main()
{
    HeapSort getelements;
    getelements.Enter_Elements();
    getelements.Get_Output();

    int hm;
    cin>>hm;
    return 0;
}
```

4. Write a program in C++ to implement common functionalities, such as traversal, insertion, and deletion in a binary search tree.

- The following program shows how to perform common functionalities, such as traversal, insertion, and deletion in a binary search tree:

```
#include<iostream.h>
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
class TreeNode
{
public:
char infoofNode[60];
TreeNode *leftchild;
TreeNode *rightchild;
TreeNode(char i[20], TreeNode *left, TreeNode
*right)
{
strcpy (infoofNode, i);
leftchild = left;
rightchild = right;
}
};
class BST
{
public:
TreeNode *ROOTNODE;
BST();
void insertinBST(char element[20]);
void findinBST(char element[20], TreeNode
**parentNode, TreeNode **location);
void inorderTraversal(TreeNode *ptr);
void preorderTraversal(TreeNode *ptr);
void postorderTraversal(TreeNode *ptr);
};
BST::BST()
{
ROOTNODE = NULL;
}
//Insertion of a node in the binary search tree
```

```

void BST:: insertinBST(char element[50])
{
    TreeNode *tmp, *parentNode, *currentNode;
    findinBST(element, &parentNode, &currentNode);
    if(currentNode != NULL)
    {
        cout<<"\nYou are not allowed to enter the words
        you have entered once";
        return;
    }
    else
    {
        tmp = new TreeNode(element, NULL,NULL);
        if(parentNode== NULL)
            ROOTNODE = tmp;
        else
            if(strcmp(element, parentNode->infoofNode)<0)
                parentNode->leftchild = tmp;
            else
                parentNode ->rightchild = tmp;
        }
    }

void BST::findinBST(char element[50],TreeNode
    **parentNode, TreeNode **currentNode)
{
    *currentNode = ROOTNODE;
    *parentNode = NULL;
    while ((*currentNode != NULL) &&
    (strcmp((*currentNode)->infoofNode, element) != 0))
    {
        *parentNode = *currentNode;
        if(strcmp(element, (*currentNode)
        >infoofNode)<0)
            *currentNode = (*currentNode)->leftchild;
        else
            *currentNode = (*currentNode)->rightchild;
    }

}

//Performing the inorder traversal
void BST::inorderTraversal(TreeNode *ptr)
{

```

```

        if(ROOTNODE==NULL)
        {
            cout<<"There are no elements in the tree and
            tree is empty";
            return;
        }
        if(ptr!= NULL)
        {
            inorderTraversal(ptr->leftchild);
            cout<<ptr->infoofNode<<" ";
            inorderTraversal(ptr->rightchild);
        }
    }
    //performing the preorder traversal
    void BST::preorderTraversal(TreeNode *ptr)
    {
        if(ROOTNODE==NULL)
        {
            cout<<"There are no elements in the tree and tree
            is empty";
            return;
        }
        if(ptr!= NULL)
        {
            cout<<ptr-> infoofNode<<" ";
            preorderTraversal(ptr->leftchild);
            preorderTraversal(ptr->rightchild);
        }
    }
    //Performing the postorder traversal
    void BST::postorderTraversal (TreeNode *ptr)
    {
        if(ROOTNODE==NULL)
        {
            cout<< "There are no elements in the tree and
            tree is empty";
            return;
        }
        if(ptr!=NULL)
        {
            postorderTraversal(ptr->leftchild);
            postorderTraversal(ptr->rightchild);
            cout<<ptr->infoofNode<<" ";
        }
    }

```

```
}  
  
}  
  
void main()  
{  
    BST b;  
    char choice, words[50];  
    int num;  
    while(1)  
    {  
        cout<<"\nSelect from the following options";  
        cout<<"\n1. Select this option to implement  
            insertions"<< endl;  
        cout<<"2. Select this option to complete  
inorder traversal of binary search tree"<<endl;  
        cout <<"3. Select this option to perform  
preorder traversal of binary search tree  
<<endl;  
        cout<<"4.Select this option to perform  
postorder traversal of binary search tree"<<  
endl;  
        cout<<"5. Select this option to exit"<< endl;  
        cout<<"\nEnter the option of your choice (1  
5):";  
        cin>> choice;  
        switch(choice)  
        {  
            case '1':  
            {  
                cout<<"Enter a word of your choice: ";  
                cin>> words;  
                b.insertinBST(words);  
            }  
  
            break;  
  
            case '2':  
            {  
                cout<<"\n";  
                b.inorderTraversal(b.ROOTNODE);  
                cout<<"\n";  
            }  
  
            break;  
            case '3':  
            {
```

```

        cout<<"\n";
        b.preorderTraversal(b.ROOTNODE);
        cout<<"\n";

    }
    break;

    case '4':
    {
        cout <<"\n";
        b.postorderTraversal(b.ROOTNODE);
        cout<<"\n";

    }
    break;

    case '5':
    {
        exit(0);
    }
    break;

    default:
    {
        cout<<"you have entered an invalid Option:" <<
endl;
        break;
    }

    }
}
}

```

5. Write a program to show how to pass parameters by value to a function.

- The following program shows how to pass parameters by value to a function:

```

#include <iostream.h>
#include <string>
using namespace std;

string EmployeeNames()
{
    string FN, LN, CompleteName;
    cout << "The first name of the Employee is: ";
}

```

```
    cin >> FN;
    cout << "The Surname of the employee is:";
    cin >> LN;

    CompleteName = FN + " " + LN;

    return CompleteName;
}

int main()
{
    string NameIncludingSurname;
    double TotalHoursWorked;

    double GetHours(string NameIncludingSurname);

    NameIncludingSurname = EmployeeNames();

    TotalHoursWorked =
        GetHours(NameIncludingSurname);

    cout << "\nName of the employee including
    surname is: " << NameIncludingSurname;
    cout << "\nTotal number of hours worked in
    office during the week is: " << TotalHoursWorked
    << " hours\n\n";

    return 0;
}

double GetHours(string NameIncludingSurname)
{
    double Monday, Tuesday, Wednesday, Thursday,
    Friday, Totaltimeworked;
    cout << endl << NameIncludingSurname <<
    Totaltimeworked;
    cout << " Duration of time worked on Monday: ";
    cin >> Monday;
    cout << "Duration of time worked on Tuesday: ";
    cin >> Tuesday;
    cout << "Duration of time worked on wednesday:
    ";
    cin >> Wednesday;
    cout << "Duration of time worked on Thursday:
    ";
    cin >> Thursday;
```

```

    cout << "Duration of time worked on Friday: ";
    cin >> Friday;

    Totaltimeworked = Monday + Tuesday + Wednesday
+ Thursday + Friday;

    cout <<"Total hours worked by" << " "
<<NameIncludingSurname << " " << "is" <<" "
<<Totaltimeworked;

}

```

6. Write a program to show different techniques to pass arguments: by value, by reference, and as pointers.

- The following example shows different techniques through which arguments can be passed. It includes passing by value, passing by reference, and passing values in the form of pointers:

```
#include<iostream.h>
#include<conio.h>
void main()
{
clrscr();
int positivenumbers = 5, negativenumbers = 4;
void PassByValue(int positive, int negative);
void Reference(int &p, int &n);
void Pointers(int *k, int *l);

cout<<"\n Displaying the number of positive and
negative numbers";
cout<<"\n\tPositive numbers =" << positivenumbers;
cout<<"\n\tNegative numbers =" <<negativenumbers;
cout <<"\nPassing the parameters by values";
PassByValue(positivenumbers, negativenumbers);
cout<<"\tGetttng the output";
cout<<"\n\tPositive number =" << positivenumbers;
cout<<"\n\tNegative number =" <<negativenumbers;
cout<<"\tPassing arguments by reference";
Reference(positivenumbers, negativenumbers);
cout<<"\nAfter calling as Reference, values are";
cout<<"\n\tPositive number ="<<positivenumbers;
cout<<"\n\tNegative number=" << negativenumbers;
cout<<"\n Passing arguments in the form of
```

```

pointers";
//Pointers(&k, &l);
cout<<"\nAfter calling as pointers, values are";
cout<<"\n\tPositive number =" <<positivenumbers;
cout<<"\n\tNegative number=" <<negativenumbers;
cout<<"\n";
int m;
cin>> m;
}
void PassByValue(int c, int d)
{
c+=3, d+=5;
cout<<"\nPass by value";
cout<<"\n\tPositive number="<< c;
cout<<"\n\tNegative number="<<d;
}
void Reference(int &c, int &d)
{
c = c + 4, d = d +5;
cout <<"\Pass by Reference";
cout<<"\n\tPositive number="<<c;
cout<<"\n\tNegative number="<<d;

}
void Pointers(int *c, int *d)
{

*c = 44, *d = 52;
cout<<"\nPassing as pointers";
cout<<"\n\tPositive number="<<*c;
cout<<"\n\tNegative number = "<<*d;

}

```

7. Write a program to calculate greatest common divisor of two numbers.

- The following program shows how to calculate the greatest common divisor of two numbers:

```

#include <stdio.h>
#include <conio.h>
#include <string>

int calculateGCD(int c, int d)

```

```

{
    int rem;
    while( d != 0 )
    {
        rem = c % d;
        c = d;
        d = rem;
    }
    return c;
}

int main()
{
    clrscr();
    int p, q;

    cout << "This program calculates the greatest
common divisor of two numbers\n";
    cout << "Please enter the first number: ";
    cin >> p;
    cout << "Please enter the second number: ";
    cin >> q;

    cout << "\n The greatest common divisor of
provided two numbers "
    << p << " and " << q << " is "
    << calculateGCD(p,q) << endl;

    cin>> p;
    return 0;
}

```

8. Write a program to calculate the total area and the total volume of a cube. The user should be asked to enter the length of side of the cube.

- The following program calculates total area and volume of a cube:

```

#include <iostream.h>
#include<Math.h>
#include<conio.h>
class PropertiesofCube
{
public:

```

```
void SideofCube(double Sd);
double GetSideofCube();
double AreaofCube();
double VolumeofCube();
void Propertiesshown();
private:
double Side;
};

void PropertiesofCube::SideofCube(double Sd)
{
    Side = Sd < 1;
}

double PropertiesofCube::GetSideofCube()
{
    return Side;
}

double PropertiesofCube::AreaofCube()
{
    return 6 * Side * Side;
}

double PropertiesofCube::VolumeofCube()
{
    return Side * Side * Side;
}

void PropertiesofCube::Propertiesshown()
{
    cout << "Following are the configuration of this
    cube";
    cout << "\nLength of the side of the cube is = " <<
    GetSideofCube();
    cout << "\nArea of this cube = " << AreaofCube();
    cout << "\nThe volume of this cube = "
    << VolumeofCube() << "\n\n";
}

void main()
{
    clrscr();
```

114 C++ INTERVIEW QUESTIONS

```
PropertiesofCube c;  
c.SideofCube(-32.45);  
c.Propertiesshown();  
  
PropertiesofCube P;  
P.SideofCube(98.54);  
P.Propertiesshown();  
  
int v;  
cin>> v;  
}
```

The result appears as follows:

```
Following are the configuration of this cube  
Length of the side of the cube is = 1  
Area of this cube = 6  
The volume of this cube = 1  
  
Following are the configuration of this cube  
Length of the side of the cube is = 98.54  
Area of this cube = 58260.7896  
The volume of this cube = 956836.367864
```

11

Friends

1. Explain the concept of friend function in C++.

- The **friend function** in C++ refers to a class or function that works with the private or protected members of the other class. It gets privileges to access and work with the private members of the other class. The programmer has full control over both the **friend** and member functions of the class and can use the features of both. The **friend** function does not require the use of an object and can be invoked without creating the object.

2. Do you agree with the saying 'friends violate encapsulation'?

- According to some developers, the friend function breaks the encapsulation rule, but some other developers believe that the friend function do not break encapsulation, rather it extends and enhances encapsulation.

Let's understand this by a situation in which a need arises to split a class into two halves. These two halves used to be in the same class and need direct access to each other as well as from a data structure. The easiest and safest way to implement this task is to make the two halves friends of each other. If the two classes are created as friends, then private members of the classes can be shared in an efficient manner.

3. **The friend functions are very useful for accessing the private members of a class. However, they also have some demerits. Can you point out the advantages and disadvantages associated with friend functions?**

- The following are the advantages of using friend functions:
 - a. A function that is declared as a friend has the privileges to access the private and protected members of the class, although friends do not come under the member function category.
 - b. The friend function can be made private or public.
 - c. Unlike a member function, the friend function can be called without the use of dot (.) operator, in the form of f(obj).
 - d. The friend function is used when two or more classes need frequent sharing of data between each other.

The following are the disadvantages of using friend functions:

- a. Unlike member functions, friend functions get added to the global namespace of the class
- b. The friend function do not follow the inheritance feature and are not inherited
- c. The friend functions do not bind at run time

4. **In context of C++, elaborate the statement “friendship isn’t inherited, transitive, or reciprocal”.**

- In the context of C++, the statement “**friendship isn’t inherited, transitive, or reciprocal**” specifies various properties, access privileges, and features shown by friends.

Let’s understand the above statement by taking an example in which three classes are defined. Consider a base class named ‘**Mybaseclass**’ which is declared as friend of some other class named ‘**Friendclass**’. **Mybaseclass** contains a child class named ‘**Myderivedclass**’. In the above example, **friendship**

isn't inherited signifies that the **Myderivedclass** class cannot access the private members of the **Friendclass** class.

Friendship isn't transitive refers to the access privileges granted to the classes that are not transitive. Let's take a new class named '**Adminclass**' which is friend to **Mybaseclass**. It does not guarantee that **Adminclass** will also be a friend of **Friendclass**.

friendship isn't reciprocal signifies that if a class is a friend of another class, it is not necessary that the other class will also be a friend of that very class.

5. **How the programmer of a class should decide whether to declare member function or a friend function?**

- A programmer should analyze the situations or requirements for deciding whether to declare the member function or friend function for a class. When a function is declared as a friend, the function is able to access all the private and protected member of the class. A member function can be used with encapsulation point of view as friend functions violate the encapsulation and can also access all the private members although they are not a part of the class. From the security point of view, use of member function is safer than the friend function.

6. **How can you provide printing for a class using operator overloading?**

- Friend left-shift operator, **operator<<**, can be overloaded to perform the task of printing in a class. In the following program, we are using a friend function to provide printing. The **operator<<** operator returns the stream of data as output. Let's understand how to provide printing by using the following code-snippet:

```
#include<iostream.h>
#include<conio.h>
#include<stdio.h>
```

```

class abc {
public:
    friend ostream& operator<< (ostream& k, const abc& out);
private:
    int s_;
};
ostream& operator<< (ostream& k, const abc& out)
{
    return k << out.s_;
}
int main()
{
    clrscr();
    abc res;
    cout << "This is an object of abc class " <<res <<
    "\n";
    int mn;
    cin>>mn;
    getch();
}

```

7. The friend functions provide more functionalities than the **printOn()** method. However, people prefer using the **printOn()** method rather than friend functions. Explain.

- Using the **printOn()** method is not always beneficial. The main reason due to which people prefer using the **printOn()** method rather than the **friend** function is that they believe **friends** do not obey encapsulation rule. However, in reality, when **friends** are used properly they extend encapsulation and improve performance. The following code-snippet shows how you can use the **printOn()** method for printing purpose:

```

#include<iostream.h>
#include<conio.h>
#include<stdio.h>

class abc {
public:
    void printOn(ostream& j) const;
};
// Since the method is declared public, non-friend
operator can be declared.
ostream& operator<< (ostream& j, const abc& prt);
// In this program,printing is being performed

```

```

inside the printOn() method which is not recommended
void abc::printOn(ostream& j) const
{
    //printing steps
}
// operator<< calls printOn() method to perform
printing which is not recommended
ostream& operator<<(ostream& j, const abc& prt)
{
    abc d;
    d.printOn(j);
    return j;
}

```

The **printOn()** method is used to provide printing for the entire class. It is better to declare the **printOn()** method as **protected** and not **public**.

People think that using the **printOn()** method decreases maintenance cost as it does not require use of a **friend** function. This assumption is not right as the member that is called by the **top-level-function** approach does not show any benefit regarding maintenance cost. Using the **printOn()** method also makes the class more complex for the programmers who have not developed the structure of the class. The programmer gets confused when he is not allowed to use public methods of the class outside the scope of class. It should be noted that when the **printOn()** method is declared **protected** or **private**, **operator<<** should be declared as **friend**.

8. Explain how input can be provided to a class.

- Input can be provided by using friend right-shift operator, **operator>>**. The parameter passed to this operator does not have a keyword **const**. The **operator>>** returns a stream of data and helps in performing input operations while using the **while** loop or the **if** statement. The following code-snippet shows how input can be provided using **left-shift operator>>**:

```

#include<iostream.h>
#include<conio.h>

```

```
#include<stdio.h>
class abc {
public:
friend ostream& operator>> (ostream& j, abc& inp);

private:
int j_;
};
ostream& operator>> (ostream& j, abc& inp)
{
return j>> inp.j_;
}
int main()
{
clrscr();
abc mn;
cout <<"Enter an object of abc class: ";
cin >> mn;
getch();
}
```

9. Explain in brief how you can provide printing functionality for all the classes present in a program?

- The implementation of printing can be done by creating a protected virtual method. This virtual method is overridden and called from a friend operator << by all the classes to show the output. The operator << is a friend function. The binding of operator << is done at the runtime. The printing functionality is shown in the following code-snippet:

```
#include<iostream.h>
#include<conio.h>
#include<stdio.h>
class Parent {
public:
friend ostream& operator<< ( ostream& f, const
Parent& p);

protected:
virtual void printOn(ostream& f) const;
};
inline ostream& operator<< (ostream& f, const
Parent& B)
{
B.printOn(f);
return f;
}
class Child: public Parent {
protected:
virtual void printOn(ostream& f) const;
};
```

12

Miscellaneous

1. Why should we use NULL or zero in a program?

- In C++, `<cstdio>` header defines `NULL`, a global symbol that represents a null pointer. `NULL` has nothing to do with standard input/output except that some functions return a null pointer. C++ programmers do not use the `NULL` global symbol, preferring to address zero pointer values with the constant integer value `0`.

Another problem is that ignoring a `0` return could crash the system when the program tries to assign values through a zero-value pointer.

2. Is recursion allowed in inline functions?

- Syntactically, the recursion (calling of the function by itself) is allowed in inline function but practically, the inline functions and their properties do not remain inside the program. Moreover, the compiler is not sure about the depth of the recursion at the time of compilation.

3. Write about the scope resolution operator.

- The scope resolution operator (`::`) can be used to define the member functions of a program outside the boundary of a class and not within the class specifier. The global scope resolution operator, which is coded as a prefix to the variable's name (for example, `:: varname`), lets you explicitly

reference a global variable from a scope in which a local variable has the same name.

A program can use scope resolution operator (::) to bypass the override of a base class member that a derived class has overridden.

4. Write a code snippet to display an integer in a binary format.

```
A:#include <stdio.h>
#include <iostream>
#include <conio.h>
void main()
{
    int i, e, f, g, h;

    e = 40;
    f = 0*21;
    g = 'A';
    h = 'A'+ 5;
    printf("e = %hf = %hg = %h/n", e, f, g);
    printf("Ascii%g, decimal%h,hex %02x\n", h, h, h);
    printf("Here is h in binary:");
    for(i = 9; i>=0; i--){
        if((1<<i)&h)
            printf("1");
        else
            printf("0");
    }
    printf("\n");
    getch();
}
```

5. List the formats in which an int can be initialized.

- The initialization of an int can be done in the following two formats:
 - a. `intnumbers=490;`
 - b. `intdigits(490);`

6. Is it possible for a member function to delete the pointer, named this?

- It is possible for a member function to use delete this pointer but on certain conditions.

The conditions which allow the member function to use delete this are listed in the following order:

1. Make sure that the allocation of this (current) object is done through new [] operator only.
2. Make sure that the invocation of your member function done on this (current) object will be the last member function. It made the calling of the last member function by the current object easy.
3. Make sure that calling of other member functions and data members should not be done after the line of code which includes delete this.
4. Examination or comparison of this pointer with other pointers and with NULL, printing or casting it, must be avoided after using the delete this line.

7. Name the debugging methods that are used to solve problems.

- The following are the debugging methods used by C++ to solve problems:
 - *GDB* —Allows debugging of the global objects which have the reference
 - *Forte* —Used as a debugger for building the application based on high performance
 - *Visual Studio* —Traces the program's source code and supports break-points, watchpoints, and threads
 - *Tusc* —Traces the last call of the system before its termination

8. Write about the various sections of the executable image.

- The sections of an executable image are as follows:
 - a. The different sections of data, such as the section of the data which is not initialized and the initialized section of the variable of the data
 - b. The specific section of the code
 - c. The allocation of all the static variables is done in the variable section which is initialized
-

9. Discuss the possibilities related to the termination of a program before entering the main() method.

- The global variables are initialized dynamically before invoking the main() method. The process of invoking the global variables is slow. If the function is called by initialization of the global variables, then the program is terminated before entering the main() method.
-

10. What is meant by the term name mangling in C++?

- The mangled name includes tokens that identify the function's return type and the types of arguments. Calls to the function and the function definition itself are recorded in the relocatable object file as references to the mangled name, which is unique even though several functions might have the same similar name. Through name mangling, the name of the function is changed into coded and unique names or symbols which can be understood by the user. In this way, the function with the same name can be differentiated with the help of this coded language.
-

11. How a macro differs from a template?

- A macro defines the meaning for an identifier. The preprocessor replaces macro in the form of strings in the source code with values derived from the macro definition. The

most common usage for macros defines a global symbol that represents a value. A macro cannot call itself.

On the contrary, a template allows you to create generic functions that admit any data type as parameters and return a value without having to overload the function with all the possible data types. A template can call itself.

12. Write about C++ storage classes.

- The storage classes are qualifiers that are used to specify the lifetime of a variable. The lifetime of a variable relates to the portion of the program that has access to the variable. Storage class directs the compiler about how to store the variable in memory and how to access variable within the program.

The following are the different types of storage classes:

- *Auto* —Identifies the local variable as automatic, which means that each invocation of the statement block in which the variable is defined gets a fresh copy with its own memory space and with re-initialization each time.
- *Static* —Refers that the scope of a static local variable begins inside the statement block in which the variable is declared and ends when the block terminates. The variable itself retains its value between executions of the statement block.
- *Extern* —Declares a global variable in one program that can be accessed by another program. The default value of an extern variable is zero. This variable is useful in a scenario where we divide one large program into different small programs and use external variable in each small program. The main advantage of using external variable is that the complexity of a program can be reduced by separating a large program into smaller programs and

using external variable, which is shared by all the programs.

- *Register* —Refers that a variable declared with the register storage class is the same as an auto variable except that the program cannot take the variable's address. Its purpose is to allow the programmer to specify conditions under which the program's performance would be improved if certain local and automatic variables were maintained in one of the computer's hardware registers.

13. Write about storage qualifiers in C++.

- Storage qualifiers are the type qualifiers that define the nature and behavior of variables.

C++ includes three storage qualifiers:

- *Const* —A const variable is one that the program may not modify except through initialization when the variable is declared. If you take the address of a const variable, the compiler must give it a memory location. When a variable is qualified as const, the compiler prevents the program from modifying the variable's contents.
- *Volatile* —A volatile type qualifier tells the compiler that the program could change the variable. One possibility is that a variable can be changed by an asynchronous interrupt service routine. The compiler must know about such a variable.
- *Mutable* —A const member function may modify a data member only if the data member is declared with the mutable qualifier. For example-

```
#include <iostream.h>
#include <conio.h>
using namespace std;
class AValue
{
    int val;
```

```

mutable int age: // number of times the object is
reported
public:
AValue(int v) : val(v) , age(0) { }
    ~AValue()
{ cout<< "AValue: " <<val
<<" was reported " << age <<"times." ; }
void report() const;
};

Void AValue::report() const
{
age++; //modify data member even though const
cout<<val<<endl;
}
int main()
{
    const AValue aval(123);
    aval.report();
    aval.report();
    aval.report();
    getch();
    return 0;
}

```

14. Write a note on reference.

- A reference provides an alias or an alternate name for the variable. In passing alias arguments, an alias to the variable in the calling program is passed. A reference must always be initialized.

The & operator identifies a reference variable in the following example:

```

int actualint;
int &otherint = actualint;

```

These statements declare an integer, named actualint, that has another name, otherint. The references made to the actualint or otherint have the same effect, with the help of an alias.

15. Explain the process of call by reference.

- In the call by reference process, you can pass arguments to a function in two ways: using pointers and using references. In both ways, the actual original object is passed into the function, and no copy is created within the scope of the function. When you pass arguments to a function using pointer, you use the * operator; while using a reference, you use the & operator. Using the & operator, the address of variable is passed to a function. The following program illustrates call by reference using an alias:

```
#include <iostream.h>
#include <conio.h>

void add_one_refs(long & n);
void add_one_ptrs(long * n);

int main()
{
    long salary = 250000;
    cout<< "Current salary: $" << salary <<endl;
    add_one_ptrs(&salary);
    cout<< "After the first raise: $" << salary <<endl;
    add_one_refs(salary);
    cout<< "After the second raise: $" << salary
    <<endl;
    return 0;
}

void add_one_refs(long & n)
{
    n++;
}

void add_one_ptrs(long * n)
{
    (*n)++;
    getch();
}
```

The preceding code snippet shows that the user is able to change the value in the variable salary by using pointers.

16. What are the advantages of using const reference arguments in a function?

- The following are the advantages of using the const reference arguments in a function:
 - a. Protects against errors that result in altering data of a program.
 - b. Allows the processing of const and non-const actual arguments by the function. On the contrary, in the prototype, the acceptance of only non-constant arguments is done by the function without a const.
 - c. Allows the generation and usage of a temporary variable by the function appropriately.
-

17. Explain the process of creating the temporary variables with the help of the C++ compiler.

- The following are the two ways to create temporary variables. The creation of temporary variables is possible only if the function parameter is a const reference.

The following are the two ways for creating temporary variables:

- a. The actual argument is the correct type, but it

```
isn't Bvalue
double Cube(const double &Cname)
{
    Cname = Cname * Cname * Cname;
    return Cname;
}
double temp = 8.0;
double value = cube(9.0 + temp); // argument is a
expression and not a Bvalue;
```

- b. The actual argument is of the wrong type, but it can be converted to the correct type.

```
long temp = 9B;
double value = cuberoot ( temp); // long to double
conversion
```

18. What kind of problems can be solved by a namespace?

- The namespace feature is used to avoid the name collision caused due to the use of the global identifiers by multiple providers of libraries. The provider of libraries avoids such name collisions by assigning the unique namespace to the libraries. The namespace feature is a logical space which uniquely identifies a resource, such as a program or class.

The declaration of a namespace is given as follows:

```
namespace [identifier] { namespace-body }
```

19. Why do we use the using declaration?

- A using declaration specifies that all the identifiers in the namespace are available to the program within the scope of the using declaration. It makes all the namespace's identifiers available to the program in the context of their own outer scope. The principle use of the using declaration is to support standard library interfaces that are well known.

20. Write about an iterator class.

- Iterator class provides an access to the classes which are inside the containers (it holds a group of objects in an organized way). The containers include the data structure, class, and abstract data type. Each container type supports one category of iterator depending on the container's requirements. The categories are input, output, forward, bidirectional, and random access. These properties specify the behavior that the iterator must exhibit in order to support the container. Iterators can be initialized, incremented, and decremented, and their bounds can be limited to the current extent of the containers. If you can cause an iterator to be equal to another iterator by incrementing the first one, the second iterator is reachable from the first. The two iterators are also known to refer to the same container. The two

iterators can therefore define a range of objects in the container.

21. Write about the stack unwinding.

- Stack unwinding is a process in which a destructor is invoked in a particular program for destroying all the local objects in the stack between throwing and catching of an exception.

22. How is the pointer hopping implements the length of a string?

- Pointer hopping is a process in which a node refers to another node by following the next reference. The following code snippet shows an example of pointer hopping:

```
#include <iostream.h>
#include <string>
using namespace std;
//A variable length 4is implemented using indexing.
intlength4( const char str3[ ] )
{
    int i;
    for( i = 0; str3[ i ] != '\0'; i++ )
        ;
    return i;
}
// length implemented with pointer hopping.
intlength6( const char *str3 )
{
    const char *Sp = str3;
    while( *Sp++ != '\0' )
        ;
    returnSp- str3 - 1;
}

int main( )
{
    char str3[616];
    cout<< "Enter strings: use END OF FILE-marker to
    terminate: " <<endl;
    while(cin>> str3 )
    {
        if(length 4( str3 ) != length6( str3 ) )
            cerr<< "Oops!!!" <<endl;
    }
}
```

```

    }
    return 0;
}

```

24. Write about the inline function.

- The inline function is a function that the compiler compiles as an in-line code every time the function is called. An inline function informs a compiler to compile a new copy of the function, each time it is called. You should use inline functions only when functions are small or when there are relatively few calls to them. The advantages of an inline function are:
 1. It offers an improved macro facility.
 2. By using the inline functions, the user can split a large function with many nested modules of statement blocks into many small inline functions. It affects a program's readability without using overhead calls of the function.

25. Name the features supported by name mangling in C++.

- The mangled name supports the use of prototypes to ensure that the functions match their calls. The formation of a unique name is done by encoding the types of parameter with the function or method name. For example:

```

int b (void) { return 1; }
int b (int) { return 0; }
void c (void) { int i = b(), e = b(0); }

```

On applying name mangling, the preceding code will change into:

```

int __b_t (void) { return 1; }
int __b_y (int) { return 0; }
void __c_t (void) { int i = __b_t(), e = __b_y(0); }
}

```

26. In which situation the program terminates before reaching the breakpoint set by the user at the beginning of the main() method.

- In C++, the initialization of variables is allowed before the calling of the main() method. If a function is invoked by the

global variable's initialization and termination is done, then it results in the error before entering of the `main()` method.

27. Write about the declaration of the following terms:

- a. Virtual member functions**
- b. The static member variable**
- c. The static function**
- d. The static destructor**

- a. *Virtual member functions* —In C++, a virtual function is a function that is declared as virtual in a base class and is redefined by a derived class. To declare a function as virtual, its declaration is preceded by the keyword `virtual`. A function is declared virtual because its execution depends on the context (contents of the base class pointer that are used to invoke it), which is not known at the time of compilation. This is also called dynamic binding.
- b. *The static member variable* —A variable's value is maintained in between the calls of different functions with the help of a static keyword. On accessing the static variable, if its value changes, it displays the changed value. If the accessing of a static variable is done again, it still holds the old data due to the allocation of a separate memory area by the compiler for storing the old data. Its biggest advantage is that the memory is updated with any modification done with the static variable. Moreover, the compiler can monitor the values of the static variables.
- c. *The static function* —Usually, you use a static member function when you need to access only the static data members outside the class. It can be invoked even when the instantiation of a class is not done. A virtual keyword cannot be used for the static member function. Static member function do not have this pointer.

- d. *The static destructor* —A static destructor is defined with the syntax `static ~this()`. A static destructor is called on terminating the program, but only after the successful execution of the static constructor. The static destructors contain empty lists of parameter. The order of execution of the static destructors is reverse as compared with the execution of the static constructors.

28. If i is an integer variable, which is faster — ++i or i++?

- In ++i which is a pre-increment method, there is only one instruction which is needed for incrementing the variable for returning a new value. In the case of i++ which is a post-incremental method, there is a need for two instructions—one for saving the old compiler which is used in the expression and the other for the incrementation of the variable. In the post-incremental method, first the old value is returned and then the variable is incremented. This process is slower than the pre-incremental process.

29. Without the use of the sizeof() function, how can you find the size of an integer data type? Demonstrate it with the help of a program.

- ```
#include<iostream.h>
int main()
{
 int *p ;
 int *q = p + 1;
 cout<< " size of an integer variable p" <<endl;
}
```

---

**30. How can I disable the “echo” feature?**

- The disabling of the echo feature is done with the help of the `getpass()` function. The character keys are turned off by using the `getpass(3)` function.

### 31. Why cannot a file be opened in a different directory such as ...\\test.dat?

- In C++, the backslash ("\") is used for identifying the special characters such as \t for tab, \n for a new line, and \b for a backspace. A file with a combination of backslashes seems to be inappropriate. For example, \\D:\\Books\\Simple Steps\\VB 2010 in SS test.dat

For a filename, a forward slash (/) should be used. The forward slash (/) has certain advantages which are:

1. It can work on all versions of Unix, such as Plan 9, all windows, and OS/2.
2. It can also work with the \ as a directory separator. Therefore, it is safer to use forward slash.

For example, C:/Program Files/Borland/Common Files/test.dat

A double back (\\) slash can also be used but it is not preferred in C++. The reason is that

if a \ is forgotten by the programmer, an error can occur.

Forexample-"D:\\Books\\Comdex\\\\.NET 4.0 Comdex\\test.dat"

### 32. Write about the opening of a stream in binary mode.

- To open a stream in the binary mode, the use of std::ios::binary is preferred. To open a binary mode, you need to include the ios::binary value in the open mode arguments for the open function or in the constructor's initializer list.

Let's understand this concept with the help of the following code snippet to read a file in binary mode:

```
#include <iostream.h>
#include <fstream.h>
#include <conio.h>
int main()
{
 char* fname = "test1.dat";
 //---read the file into an array
 ifstream tfile(fname, ios::in | ios::binary);
 ostream ofile(tfile.rdbuf());
 char tdata[100];
 int i = 0;
```

```

while(!tfile.eof() && i < sizeof tdata)
 tfile.get(tdata[i++]);
tfile.seekg (0, ios::end);
ofile<< "\r\n";
for (int j = 0; j < i-1; j++)
 ofile.put(static_cast<char>(tdata[j]));
for (int j = 0; j < i-1; j++)
 cout<<tdata[j];
getch();
return 0;
}

```

The program modifies the file to include the contents. The preceding program sends the string "\r\n" to the file as a line separator instead of the endl manipulator or the newline character.

---

**33. Write down the steps of reopening the std::cin and std::cout in the binary mode.**

- The reopening of the std::cin and std::cout depends upon their implementation. For example, in case of binary input/output using std::cin and std::cout, there is no such thing with a particular standard to confirm about the opening of std::cin and std::cout in the binary mode.

Moreover, an attempt to reopen them in the binary mode by closing the streams results in undefined behavior and errors. C++ has no input/output operators as integral parts of the language. Therefore, to extend the language with input/output functions, C++ depends on class libraries for its input and output.

# 13

## C and C++

---

### 1. How can you link a C program with a C function?

- A C program can be linked with a C function with the help of the extern "C" linkage specification. The name mangling in C++ encodes all the codes into the symbol and therefore results in errors. Therefore, the references of these symbols should be avoided by the C compiler. One should have the knowledge about the mangled function and type-safe linkages. During the compilation, with the extern "C" linkage specification, the name mangling feature is turned off to ensure the proper linkage of the C program to the C functions.

---

### 2. How are the features of C++ different from C?

- All the features of C are similar to C++ except some features, such as polymorphism, inheritance, operator overloading which are supported in C++ but not in C language. Both C and C++ language is similar in their functionality but C++ provides with more tools and options.

---

### 3. How can a struct in C++ differs from a struct in C?

- The differences between struct in C++ and C are listed in the following points:
  1. In C and C++, the variables of the structures are public; however, in C, the variable cannot be declared as private or protected. On the contrary, in C++, the variables can be declared as private or protected.

2. On declaring a struct in C, the addition of the struct keyword is must. On the contrary, there is no need of the struct keyword on declaring struct in C++.
3. In C, structures do not have direct functions or methods (procedures). Example, pointers inside the functions. On the other hand, C++ has direct functions or methods, for example classes.
4. In C, the initialization cannot be done outside the scope of a structure. However, in C++, the initialization can be done outside the scope of a structure.
5. In C, the concept of inheritance is not supported. In C++, the concept of inheritance is fully supported.

---

**4. What happens when the extern "C" char func (char\*,waste) executes?**

- On executing the extern "C" char func (char\*,waste), the link to the code compiled by the C compiler can be established by the programmer because the "name mangling" (which includes the tokens that identify the function's return type and the types of its arguments) will be turn off for the function.

---

**5. Write about the access privileges in C++ and also mention about its default access level?**

There are three access privileges provided in C++ which are private, protected, and public. The members of a class have private access by default. The public members can be accessed by member functions and by other functions that declare an instance of the class. The private members can be accessed by member functions of the same class. The protected members are included by using the class inheritance. The protected members can be accessed only within the class.

---

**6. Describe the role of the C++ in the tradeoff of safety vs. usability.**

- Earlier in C, encapsulation is meant to hide the information. Encapsulation is included in C by making other class members static in a class to prevent accessing the members by another module. However, the concept of multiple instances is not supported by the encapsulation because it is impossible to directly make multiple instances inside the static class module. If there is a need of multiple instances, a struct is used. On the contrary, encapsulation is not supported by structs.

In C++, a class supports both encapsulation and multiple instances. The class's interface that contains the public member functions and friend functions of the class can be accessed by multiple users. On the other hand, an implementation of the class is defined by the private and protected parts of a class. This encapsulated part forms a struct; thereby, reducing the tradeoff (loosing of one quality as a result of gaining of another quality) between the encapsulation and usability.



## Question Index

### **Chapter 1: Constructors / Destructors**

1. Define what is constructor or ctor. 1
2. Can we define a constructor as virtual in C++? 1
3. Can we declare a base-class destructor as virtual? 2
4. How does a copy constructor differs from an overloaded assignment operator? 2
5. Define two methods of constructor invocation. 2
6. Define the process of error-handling in case of constructor failure. 3
7. Explain the concept of copy constructor. 3
8. What is a default constructor? 3
9. When are copy constructors called? 3
10. Is it possible to pass an object of the same class in place of object reference to the copy constructor? 4
11. Define a conversion constructor. 5
12. What is a conversion operator? 6
13. Define the process of handling in case of destructor failure. 7
14. What is a virtual destructor? 8
15. What is the sequence of destruction of local objects? 8
16. Write about the sequence in which objects in an array are destructed. 8
17. Is it possible to overload the destructor of a class? 9
18. Can you explicitly call a destructor on a local variable? 9
19. What happens if a local variable is destroyed within its scope before a program is completely executed? Can a destructor be called if needed? 9

## 142 C++ INTERVIEW QUESTIONS

20. Explain how would you handle a situation where you cannot call the destructor of a local explicitly? 10
21. What happens if the local variable cannot be wrapped in an artificial block? 10
22. What are shallow and deep copies? 11
23. What do you understand by zombie objects in C++? 11
24. Can a destructor be explicitly called if an object is created using the new operator? 12
25. Explain the use of the placement new method. 12
26. On implementing a destructor, is there any need for the explicit calling of the destructors for member objects? 14
27. On implementing a destructor of the derived class, is there any need for the explicit calling of the destructor for my base class? 15
28. How does List r; differs from List r();? 16
29. Can one constructor of a class be called by another constructor of the same class for initializing the this object? 16
30. Is the default constructor for a class Bred always Bred::Bred()? 18
31. On creating an array of bird\_class objects, which constructor will get called? 19
32. Can we use initialization lists or assignments in a constructor? 20
33. Should the this pointer can be used in the constructor? 21
34. What do you mean by the term named constructor? 21

### **Chapter 2: Virtual Functions**

1. What is a virtual function? 25
2. Define pure virtual function. 26
3. What do you understand by a pure virtual member function? 26
4. How can virtual functions in C++ be implemented? 27

5. What do you understand by pure virtual function? Write about its use? 27
6. How the virtual functions maintain the call up? 27
7. Write a note on virtual destructor. 28

**Chapter 3: Inheritance**

1. Write a note about inheritance. 29
2. What is the need of multiple inheritance? 29
3. Write about the IS A and HAS A class relationships and also explain about their implementation in a class. 30
4. Explain the concept of template. 30
5. How can you say that a template is better than a base class? 31
6. Explain the concept of multiple inheritance (virtual inheritance). Write about its advantages and disadvantages. 31
7. Describe inheritance and non-inheritance of a derived class. 32
8. Write a note about the virtual member function. 32
9. How static typing is achieved by dynamic binding in C++? 32
10. How calling of virtual member functions differ from that of non-virtual member functions? 33
11. Write about the performance cost of using virtual functions in C++. 34
12. Write about the functional overriding of the base class by the member function in the derived class. 36
13. Explain the concept of dynamic binding with the help of an example. 37
14. When should a destructor be made virtual? 38
15. Should the member functions which are made public in the base class be hidden? 38
16. Write a note about upcasting. 39

## 144 C++ INTERVIEW QUESTIONS

17. Can a derived array be called a kind-of-base array? 39
18. Can an array be replaced with a template container class? 40
19. Can circle be called an ellipse? 40
20. Why do we separate interface from implementation? 41
21. How can you differentiate between inheritance and implementation in C++? 41
22. Write about abstract base classes. 42
23. Write about pure virtual member function. 42
24. Explain about a copy constructor or an assignment operator's declaration which contains a pointer to a (abstract) base class? 44
25. Write about private inheritance. 45
26. Write about different relationships that exist with the private inheritance. 46
27. Which is more preferable between composition and private inheritance? 47
28. Write about void pointers and its declaration. 50
29. Write about the relationship between the private and protected inheritance. 50
30. Which should be more useful: the protected and public virtuals? 51
31. Describe the setting up of my member functions to avoid overriding by the derived class. 51
32. Write about the rules of accessing the private and protected inheritance. 52
33. Write a note on dreaded diamond. 52
34. Write about "delegate to a sister class" through virtual inheritance. 53

**Chapter 4: Polymorphism**

1. Explain polymorphism with the help of an example. 55
2. Write about the issues related to the runtime type identification. 55

**Chapter 5: Classes and Objects**

1. What is a class? 57
2. How do C++ struct differs from the C++ class? 57
3. Write about the use of the virtual destructor. 58
4. Write a note on encapsulation. 58
5. Write a note about this pointer. 59
6. What are the effects after calling the delete this operator? 59
7. Write a note about assignment operators. 60
8. Write about all the implicit member functions of a class? 60
9. Write about the container class and mention its types. 61
10. What is overriding. 62
11. How do you access the static member of a class? 62
12. Write about a nested class and mention its use. 64
13. Write about the local class and mention its use. 64
14. Write about the members that a derived class can add. 65
15. Describe the process of creation and destruction of a derived class object. 65
16. Explain the process of creating a subscript operator for a matrix class. 65
17. Give reason why the interface of the MyMatrix class should not look similar to array-of-array. 67
18. Write about the use of the static data members. 67
19. What is an object? 70
20. Define a good interface. 70

## 146 C++ INTERVIEW QUESTIONS

21. Write about the role of C++ in the tradeoff of safety vs. usability. 70
22. How can you prevent accessing of the private parts of my class by other programmers (violating encapsulation)? 71
23. Can non-public members of another instance of the class be retrieved by the method of the same class? 71
24. Can encapsulation be called as a security device? 71
25. How the keyword struct is different from the keyword class in C++? 72

### **Chapter 6: Memory Allocation/Deallocation**

1. How new/delete differs from malloc()/free? 73
2. How the delete operator differs from the delete[] operator? 73
3. How a new operator differs from the operator new? 73
4. Explain the term memory alignment. 74
5. Explain the concept of dynamic allocation of memory. 74
6. In the expression, `char *c = (char *)malloc(20);` followed by `free(r);` explain the role of `malloc()` operator in allocating the number of bytes? 75
7. How the allocation of two-dimensional arrays is done by using new? 75
8. Is it possible to use the free operator with the new allocated pointers and the delete operator() with the malloc allocated pointers? 76
9. Can a new be used in place of old malloc()? If yes, why? 76
10. Is it possible to use a new for the reallocation of pointers ? 77
11. Is there any need for checking of NULL after executing `p = new Animal();` 77
12. Explain the process of returning of a NULL by the old compiler after the checking of new. 77

13. Can a NULL be checked before deleting the p variable from the expression, Delete p; and not if (p != NULL) ? 78
14. Enlist the processes involved in deleting a variable. 78
15. On throwing an exception by the Animal constructor in p = new Animal(), can memory leak occur? 79
16. What is the use of [] in the delete statement of the following expression: 79

```
Use p = new T[n] and delete[] p:
Animal* p = new Animal[80];
...
delete[] p;
```

17. What would happen on forgetting [], while deallocating an array through new? 79
18. Can [] be dropped on deletion of some built-in type array (char, int etc)? 79
19. Write about the retrieval of n number of objects during the process of delete[]p? 79
20. Is it possible for a member function to use delete this? 80
21. Discuss the effects occur, after an exception thrown by a member function is unspecified by an exception specification. 80
22. Explain the throw statement with its use. 81
23. How can i be thrown polymorphically? 82
24. What is the role of copy constructor in copying of thrown objects? 83
25. What is the role of Microsoft Foundation Class (MFC) in relation to encourage the use of catch-by-pointer? 83
26. How can an improvement in the quality of software be done by try/catch/throw? 84
27. Explain the process of throwing variables. 84
28. Explain the process of catching a variable. 86

### **Chapter 7: Pointers**

1. What is a dangling pointer? 87
2. Explain the concept of memory leak. 87
3. What is an auto pointer? 87
4. List the issue that the auto\_ptr object handles. 88
5. What are smart pointers? 88
6. Write about the complexities related to the following code: 89

```
int*b = NULL;
int&p = *b;
```

7. How a pointer differs from a reference? 89
8. How constint\*ourPointer differs from intconst\*ourPointer? 89
9. Mention the conditions in which we should use references and pointers separately. 90

### **Chapter 8: Strings**

1. What happens on executing the following code? 91
2. Explain the ways in which type conversion can be done. 91

### **Chapter 9: Overloading Operators**

1. List the operators that cannot be overloaded. 93
2. Explain how overloading takes place in C++. 93
3. What is the difference between prefix and postfix versions of operator++()? 93
4. Can a function overloading depend only on passing by value and passing by reference ? 94
5. What's the use of operator overloading? 94
6. Describe the advantages of operator overloading. 95
7. Provide some examples of operator overloading. 95

8. Operator overloading is supposed to make the code clearer and more understandable but it makes the code more complex to understand. Why? 95
9. Identify the operators that cannot be overloaded. Provide an example using any one of the operator. 96
10. Can the operator == be overloaded for comparing two arrays consisting of characters by using string comparison? 97
11. Can the creation of operator\*\* is allowed to perform the to-the-power-of operations? 97
12. What is the main purpose of overloading operators? 98
13. Specify some guidelines that should be followed while overloading operators? 98

**Chapter 10: Algorithms**

1. Write a code-snippet to check if the given linked list is in the ascending order? 99
2. Write a program to reverse the elements in a link list. 99
3. Write a program to implement Binary Heap in C++. 101
4. Write a program in C++ to implement common functionalities, such as traversal, insertion, and deletion in a binary search tree. 104
5. Write a program to show how to pass parameters by value to a function. 108
6. Write a program to show different techniques to pass arguments: by value, by reference, and as pointers. 110
7. Write a program to calculate greatest common divisor of two numbers. 111
8. Write a program to calculate the total area and the total volume of a cube. The user should be asked to enter the length of side of the cube. 112

**Chapter 11: Friends**

1. Explain the concept of friend function in C++. 115

## 150 C++ INTERVIEW QUESTIONS

2. Do you agree with the saying 'friends violate encapsulation'? 115
3. The friend functions are very useful for accessing the private members of a class. However, they also have some demerits. Can you point out the advantages and disadvantages associated with friend functions? 116
4. In context of C++, elaborate the statement "friendship isn't inherited, transitive, or reciprocal". 116
5. How the programmer of a class should decide whether to declare member function or a friend function? 117
6. How can you provide printing for a class using operator overloading? 117
7. The friend functions provide more functionalities than the `printOn()` method. However, people prefer using the `printOn()` method rather than friend functions. Explain. 118
8. Explain how input can be provided to a class. 119
9. Explain in brief how you can provide printing functionality for all the classes present in a program? 120

### **Chapter 12: Miscellaneous**

1. Why should we use NULL or zero in a program? 121
2. Is recursion allowed in inline functions? 121
3. Write about the scope resolution operator. 121
4. Write a code snippet to display an integer in a binary format. 122
5. List the formats in which an int can be initialized. 122
6. Is it possible for a member function to delete the pointer, named `this`? 123
7. Name the debugging methods that are used to solve problems. 123
8. Write about the various sections of the executable image. 124

9. Discuss the possibilities related to the termination of a program before entering the main() method. 124
10. What is meant by the term name mangling in C++? 124
11. How a macro differs from a template? 124
12. Write about C++ storage classes. 125
13. Write about storage qualifiers in C++. 126
14. Write a note on reference. 127
15. Explain the process of call by reference. 128
16. What are the advantages of using const reference arguments in a function? 129
17. Explain the process of creating the temporary variables with the help of the C++ compiler. 129
18. What kind of problems can be solved by a namespace? 130
19. Why do we use the using declaration? 130
20. Write about an iterator class. 130
21. Write about the stack unwinding. 131
22. How is the pointer hopping implements the length of a string? 131
24. Write about the inline function. 132
25. Name the features supported by name mangling in C++. 132
26. In which situation the program terminates before reaching the breakpoint set by the user at the beginning of the main() method. 132
27. Write about the declaration of the following terms: 133
  - a. Virtual member functions
  - b. The static member variable
  - c. The static function
  - d. The static destructor
28. If i is an integer variable, which is faster — ++i or i++? 134

## 152 C++ INTERVIEW QUESTIONS

29. Without the use of the `sizeof()` function, how can you find the size of an integer data type? Demonstrate it with the help of a program. 134
30. How can I disable the "echo" feature? 134
31. Why cannot a file be opened in a different directory such as `...\test.dat`? 135
32. Write about the opening of a stream in binary mode. 135
33. Write down the steps of reopening the `std::cin` and `std::cout` in the binary mode. 136

### **Chapter 13: C and C++**

1. How can you link a C program with a C function? 137
2. How are the features of C++ different from C? 137
3. How can a struct in C++ differs from a struct in C? 137
4. What happens when the extern "C" char func (char\*,waste) executes? 138
5. Write about the access privileges in C++ and also mention about its default access level? 138
6. Describe the role of the C++ in the tradeoff of safety vs. usability. 139

## Glossary

### **Abstract Base Classes**

It is a class definition that is always a base class for other classes to be derived from.

### **Assignment Operator**

It is defined as the full construction of the member object by the default constructor, and results in the allocation of some amount of memory by default or the opening of some default file.

### **Auto-Pointer**

When you allocate some memory to a variable and that variable becomes out of scope, then the deallocation of the memory can be done with the help of an auto pointer.

### **Call by Reference**

In this process, you can pass arguments to a function in two ways: using pointers and using references.

### **Class Template**

It is used to build general-purpose container classes, such as stacks, lists, and queues.

### **Class**

It is a prototype that defines the abstract characteristics of an object, such as attributes, fields, or properties. It also defines the behaviors of objects, such as methods, operations, or its features.

### **Composition**

It includes the different types of compositions mean class relationships, such as has-a and is-a relationship.

### **Const Reference Arguments**

It protects against errors that result in altering data of a program. It allows the processing of const and non-const actual arguments by the function.

### **Constructor**

It is defined as a method that is called when a new instance of an object of a class is created. It initializes all class members whenever you create an object of the class.

**Container Class**

It provides built-in features and procedures that let you handle stored data, find the number of stored objects, insert new objects, and so on. The container class includes general-purpose classes, such as stacks, lists, and queues in which the maintenance of the container is generic but the item in the container is specific.

**Conversion Constructor**

It is a single argument constructor. It is used by the compiler to convert a type of object as an argument to a class type.

**Conversion Operator**

It converts an object of a class in which you define the function to an object of a different data type.

**Copy Constructor**

It is a special type of parameterized constructor. It copies one object to another. It is called when an object is created and equated to an existing object at the same time.

**Dangling Pointer**

When the location of the deallocated memory is pointed by the pointer even after the

deletion or allocation of objects is done, without the modification in the value of the pointer, then this type of pointer is called a dangling pointer.

**Deep Copy**

It is referred to a copy of the dynamically allocated objects is created with the help of a deep copy.

**Default Constructor**

It is a zero-argument constructor or a constructor in which all the arguments have default values.

**Delete Operator**

It is referred to the deletion of memory you are no longer using which clutters up the program.

**Dreaded Diamond**

The appearing of a particular class more than once in a hierarchy of class inheritance is known as dreaded diamond.

**Dynamic Allocation**

Allocating the memory dynamically means that you can allocate space for as many elements as you need at runtime. The memory can be allocated dynamically with the

new operator and free with the delete operator.

### **Dynamic binding**

It ensures that the function calls are resolved at run time; thereby, providing users the flexibility to alter a call without having to modify the code.

### **Dynamic\_Cast Operator**

For using the `dynamic_cast`, the instantiation of an object is must. Furthermore, `dynamic_cast` works only with pointers and references to polymorphic class objects.

### **Encapsulation**

It is an object-oriented design approach that closely binds the implementation of a class to its data representation, logically hides the details of the implementation from users of the data type, and provides a public interface to the data type's behavior.

### **Extern "C"**

During the compilation, with the `extern "C"` linkage specification, the name mangling feature is turned off to ensure the proper linkage of the C program to the C functions.

### **Friend function**

In C++, friend function refers to a class or function that works with the private or protected members of the other class. It gets privileges to access and work with the private members of the other class.

### **Function Overloading**

It helps in defining more than one function with the same name, but with different signatures. An error is raised if two overloaded functions are provided with the same function signature.

### **Function Template**

It is defined as a function using an abstract class, non-specific data types and then uses that template to create functions that use specific data types, such as `int` or `float`.

### **HAS A Relationship**

Inheritance is not appropriate for relationships where one class has an instance of another. For example, an employee has a date of birth and a department has a manager. This relationship is called the HAS A relationship.

**Implementation**

It is a private member of a class. The implementation defines the details of how the class implements the behavior of the abstract base type.

**Inheritance**

It is a mechanism that allows the object of one class to utilize the form and model or behavior of another class. The inherited class is said to be derived from the base class.

**Inline Function**

It is a function that the compiler compiles as in-line code every time, the function is called.

**Interface**

It is the public member of a class, which define the class user's interface to the class's data and its behavior.

**IS A Relationship**

A car is a kind of vehicle. An engineer is a kind of employee, which is a kind of person. This relationship is called the IS A relationship.

**Iterator Class**

It provides an access to the classes which are inside the containers (it holds a group of objects in an organized way).

**Leaf Method**

With the help of the leaf method, it is possible to leave the code unaligned at the time of execution by simply adding a comment next to the method.

**Linked List**

This class constructs and destroys the objects of a class. Its public interface consists of a constructor and destructor, member function to append, insert and remove objects of the type and member functions to navigate the list.

**Local Class**

It is defined as the abstract characteristics of an object, such as attributes, fields, or properties.

**Macro**

It defines the meaning for an identifier. The preprocessor replaces macro in the form of strings in the source code with values derived from the macro definition.

**Malloc() Operator**

The memory uses the malloc() operator, which in turn uses the free operator, to remove the unnecessary programs.

**Memory Alignment**

In C++, there is a requirement of setting of various objects and variables in a particular way in the system's memory. Therefore, many data variables are aligned automatically by the compiler according to their processor and type.

**Memory Leak**

When a variable does not exist longer in the memory, and deletion or reuse of that variable cannot be done, then its destruction occurs automatically. This concept is called memory leak.

**Microsoft Foundation Class (MFC)**

It implements the applications on a particular operating system platform, such as Microsoft windows. MFC uses the backward-compatibility in relation to exception handling.

**Multiple Inheritance**

It allows you to define a new class that inherits the characteristics of several base classes which are not related to each other.

**Named Constructor**

It is defined as the declaration of all the class's constructor in

the private or protected sections and declaration of public static methods for returning an object becomes possible.

**Namespace**

It is used to avoid the name collision caused due to the use of the global identifiers by multiple providers of libraries.

**Nested Class**

It is a class, defined within the scope of another class.

**New Operator**

It returns a pointer to the newly allocated memory.

**Non-Inheritance**

There are few methods that are not inherited on creating a derived class which includes constructors, destructors, and assignment operator methods of their base classes.

**Non-Virtual Member Function**

The calling of the non-virtual member function depends on the selection of a member function at the compile time based on the type of a pointer or reference to the object.

**NULL**

In C++, `<cstdio>` header defines NULL, as a global

symbol that represents a null pointer.

### **Object**

It is a declared instance of a data type, which includes standard C++ data types as well as objects of classes.

### **Operator <<**

It can be overloaded to perform the task of printing in a class. In the following program, we are using a friend function to provide printing.

### **Operator Overloading**

It helps in giving special meanings to operators, when they are used with user-defined classes.

### **Overload Assignment Operator**

Assigns the value of an existing object of the same class to another existing object in that class.

### **Overriding**

To override a method, a subclass of the class that originally declared the method must declare a method with the same name, return type (or a subclass of that return type), and same parameter list.

### **Performance Cost**

It means that when a class has at least one virtual function, the compiler builds a table of virtual function pointers for that class.

### **Placement New**

It places an object at a particular location in memory. For placing an object to its particular location, it allocates a pointer parameter to a new part of a new expression.

### **Pointer Hopping**

It is a process in which a node refers to another node by following the next reference.

### **Pointer**

A variable that can contain the address of functions or other variables. The item pointed to can be referenced through the pointer.

### **Polymorphism**

It is defined as the exhibition of a unique behavior or response by different objects when responding to the same message.

### **PrintOn () method**

It is used to provide printing for the entire class. It is better to declare the printOn ()

method as protected and not public.

### **Private Inheritance**

When you use private inheritance, the public and protected members of the base class become private members of the derived class.

### **Private Members**

They can be accessed by member functions of the same class.

### **Protected Inheritance**

When you derive a base class using protected access specifier, the private members of the base class become protected members of the derived class (the private members of the base class are not available to the derived class).

### **Protected members**

They are included by using the class inheritance. The protected members can be accessed only within the class.

### **Protected Virtual Functions**

They are used for hiding some methods or data members from the outside class making it consistent and symmetrical in its approach.

### **Public Members**

They can be accessed by member functions and by other functions that declare an instance of the class.

### **Public Virtual Functions**

It permits a program to directly modify the values of data members. It can be accessible to any function that is within the scope of the structure.

### **Pure Virtual Function**

It is defined as a virtual function in a base class. It is implemented in a derived class. A program may not declare an instance of a class that has a pure virtual function.

### **Pure Virtual Member Function**

It is defined with the no body structure. You can declare a pure virtual member function by adding the notation =0 to the virtual member function declaration.

### **Recursion**

It is the ability of a function to call itself directly or indirectly from functions that it calls.

### **Reference**

It is a variable name that is an alias for another variable.

**Runtime Type Identification (RTTI)**

It is a collection of technologies that allows you to identify an item's type at runtime, which is very useful when you deal with runtime polymorphism.

**Scope Resolution Operator (::)**

It can be used to define the member functions of a program outside the boundary of a class and not within the class specifier.

**Shallow Copy**

It Copies the actual values of the data. It means, if the pointer points to dynamically allocated memory, the new object's pointer in the copy still points to items inside the old objects and the returned object will be left pointing to items that are no longer in scope.

**Smart pointers**

They are almost similar to pointers with additional features, such as automatic destruction of a variable when it becomes out of scope and the throwing of exceptions that ensures the proper destruction of the dynamically allocated objects.

**Stack Unwinding**

It is a process in which a destructor is invoked in a particular program for destroying all the local objects in the stack between throwing and catching of an exception.

**Static typing**

It ensures that the execution at run-time will only get values that are of the right type for the program. This typing detects error in the early stages of a program, minimizing the possibility of error occurrence at the run time. This helps in improving the performance of the program.

**Storage Classes**

They are the qualifiers that are used to specify the lifetime of a variable.

**Storage Qualifiers**

They are the type qualifiers that define the nature and behavior of variables.

**Structure**

It is a record format consisting of one or more objects of multiple data types.

**Template Container Class**

It includes a vector class from Standard Template Library

(STL). You can add information to and remove information from the end of the vector class.

### **Template**

It allows you to describe a generic type to manage other data types. Using templates, you can implement the algorithm once in your code and use it with many different types of data, even with objects.

### **The Name Mangling**

In C++, name mangling includes the tokens that identify the function's return type and the types of its arguments. It encodes all the codes into the symbol and therefore results in errors.

### **The postfix operator ++ ()**

It consists of a dummy parameter of int data type.

### **this Pointer**

It exists in all non-static member functions. The this pointer is a pointer to an object of the class. It points to the object for which the function is being executed.

### **Type Conversion**

It refers to the process of assigning a value of one data

type to a variable of another data type.

### **Upcasting**

It casts a pointer or a reference from a derived class to one of the base classes in the same hierarchy.

### **Using Declaration**

It specifies that all the identifiers in the namespace are available to the program within the scope of the using declaration.

### **Virtual Constructor**

It is used to declare an object of a copy constructor or an assignment operator in a class containing an abstract base class pointer. In this method, a pure virtual clone() method is declared in the base class.

### **Virtual Destructor**

It helps in destroying objects without knowing their type. With the help of virtual function mechanism, the appropriate destructor for the object is invoked.

### **Virtual function**

It is defined as a member function in a class from which other classes may be derived.

### **Virtual Member Function**

It is a member function of the base class and relies on a specific object to determine which implementation of the function is called. However, a virtual function can be declared a friend of another class.

### **Void Pointers**

It can point to any kind of variable. You can use a pointer of the void\* type to hold a pointer to any non-constant data type.

### **Vptr**

Each object of the class in memory includes a variable called the vptr, which points to the class's common vtbl.

### **V-Table**

When a class has at least one virtual function, the compiler builds a table of virtual function pointers for that class. This table, commonly called the vtbl, contains an entry for each virtual function in the class.

### **Zombie Objects**

It is defined as an object which becomes free and no more required for an application. This zombie object occupies a space in the memory and wait for their termination.



# C++

## Interview Questions



The purpose of an interview is to access the level of knowledge of a candidate on a subject and evaluate the extent to which one can contribute towards a company's development. Even after going through effective training and being well equipped with references, you might come across some questions that may put you in a fix. This book, C++ Interview Questions, prepares you to crack such kind of questions. It is highly recommended for beginners, students, as well as professionals. This book contains comprehensive content on various concepts of the Object Oriented Programming (OOP) and C++ languages, such as constructors and destructors, virtual functions, inheritance, polymorphism, classes and objects, memory allocation and de-allocation, pointers, strings, overloading operators, friends, algorithms, and other miscellaneous topics.

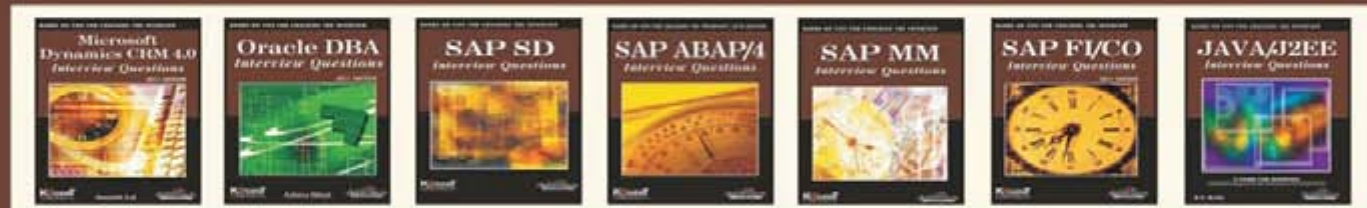
### The book covers:

- The concepts in a format that helps in learning the various aspects of constructors and destructors
- Information about virtual functions
- Questions on inheritance and concepts related to template, class template, function template, non-inheritance, and static typing and dynamic binding
- The definition of polymorphism and Runtime Type Identification (RTTI)
- Information on classes and objects, memory allocation and de-allocation, pointers and strings
- The concepts of C and C++, such as name mangling, type-safe linkages, extern C, public members, and private members
- Overloading operators and topics such as function overloading, operator overloading, the postfix operator ++ (), and pass by reference and pass by value
- Questions on algorithms, linked list, binary heap, traversal, insertion and deletion in a binary search tree, and the friend function
- Information about the concepts of inline functions, the scope resolution operator, debugging methods, C++ storage classes, storage qualifiers, temporary variables, namespace, and the Iterator class

### ABOUT THE AUTHORS

The proficient team at **Kogent Learning Solutions Inc.** and **Dreamtech Press** has seized the market of computer books bringing excellent content in software development to the fore. The team is committed to excellence—excellence in quality of content, excellence in the dedication of its authors and editors, excellence in the attention to detail, and excellence in understanding the needs of its readers.

### ALSO AVAILABLE:



978-81-7722-804-5



9 788177 228045

**dreamtech**  
PRESS

19-A, Ansari Road, Daryaganj, New Delhi-110002  
Tel.: 91-11-23243463, 23284212, 23243075  
Fax: 91-11-23243078  
Email: [feedback@dreamtechpress.com](mailto:feedback@dreamtechpress.com)  
[www.dreamtechpress.com](http://www.dreamtechpress.com)  
Blog: <http://dreamtechpress.wordpress.com>